

Ad Targeting Optimization

Unsupervised ML + Deep Learning

Objective: Discover hidden user segments from ad impression data using K-Means and Autoencoder-based clustering, evaluate cluster quality, and translate findings into data-driven ad budget recommendations. **Dataset:** `ad_click_dataset.csv` ---

Table of Contents

1. [Environment Setup & Imports]
2. [Dataset Generation / Loading]
3. [Data Preprocessing]
4. [K-Means Clustering]
5. [Autoencoder + K-Means]
6. [Model Comparison]
7. [Cluster Interpretation]
8. [Business Questions 1–9 — Answers & Analysis]
9. [Final Recommendations & Budget Reallocation]

1. Environment Setup & Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.cm as cm
import seaborn as sns
import warnings, os
import scipy.stats as ss
import tensorflow as tf
from tensorflow import keras
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score,
    davies_bouldin_score, calinski_harabasz_score, silhouette_samples
from sklearn.impute import SimpleImputer
from sklearn.manifold import TSNE

warnings.filterwarnings('ignore')
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
# Plot style
plt.rcParams.update({'figure.dpi': 120, 'axes.spines.top':
    False, 'axes.spines.right': False,
    'axes.grid': True, 'grid.alpha': 0.3, 'font.size': 11,})
COLORS = ['#185FA5', '#1D9E75', '#D85A30', '#9B59B6', '#E67E22', '#E74C3C',
    '#AAAAAA']
print("✅ All imports successful.")

✅ All imports successful.
```

Plot Styling

To ensure consistent and aesthetically pleasing visualizations throughout the notebook, a custom plot style has been defined. This style includes settings for figure resolution, grid lines, font sizes, and removes the top and right spines from plots for a cleaner look. The COLORS palette will be used in individual plotting functions to maintain visual consistency.

```
def set_plot_style(COLORS):
    """Applies a consistent plot style for the notebook."""
    plt.rcParams.update({'figure.dpi': 120, 'axes.spines.top':
        False, 'axes.spines.right': False,
        'axes.grid': True, 'grid.alpha': 0.3, 'font.size': 11,})

def apply_plot_settings(ax, colors):
    """Applies consistent visual settings to a given matplotlib Axes
    object."""
    # Common settings for axes
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.grid(True, linestyle='--', alpha=0.3)
```

2. Dataset Generation / Loading with EDA

If you have the real `ad_click_dataset.csv`, place it in the working directory and skip the generation cell below.

The synthetic generator reproduces the exact schema described in the problem statement.

```
import gdown
# SECTION : DOWNLOAD DATASET
url = f"https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/169/383/original/ad_click_dat
output = "Country-data.csv"
gdown.download(url, output, quiet=False)
```

Downloading...

From:

[https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/169/383/origin](https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/169/383/original)

To: /content/Country-data.csv

100%|██████████| 465k/465k [00:00<00:00, 11.3MB/s]

```
{"type": "string"}
```

```
# — Load dataset —————
```

```
CSV_PATH = '/content/Country-data.csv'
```

```
if os.path.exists(CSV_PATH):
```

```
    df_raw = pd.read_csv(CSV_PATH)
```

```
    print(f"✅ Loaded dataset: {df_raw.shape}")
```

```
else:
```

```
    print(f"❌ File not found: {CSV_PATH}")
```

```
df_raw.head(2)
```

```
✅ Loaded dataset: (10000, 9)
```

	id	full_name	age	gender	device_type	ad_position	browsing_history	time_of_day	click
0	670	User670	22.0	NaN	Desktop	Top	Shopping	Afternoon	1
1	3044	User3044	NaN	Male	Desktop	Top	NaN	NaN	1

```
def basic_eda(df):
```

```
    # Basic EDA
```

```
    print("Shape:", df.shape)
```

```
    print("\nData types:")
```

```
    print(df.dtypes)
```

```
    print("\nMissing values:")
```

```

print(df.isnull().sum())
print(f"\nOverall Click-Through Rate (baseline): {df['click'].mean():.4f}
      ({df['click'].mean()*100:.1f}%)")
basic_eda(df_raw)

```

Shape: (10000, 9)

Data types:

```

id                int64
full_name         object
age              float64
gender           object
device_type       object
ad_position       object
browsing_history  object
time_of_day       object
click            int64
dtype: object

```

Missing values:

```

id                0
full_name         0
age              4766
gender           4693
device_type       2000
ad_position       2000
browsing_history  4782
time_of_day       2000
click            0
dtype: int64

```

Overall Click-Through Rate (baseline): 0.6500 (65.0%)

EDA VISUALIZATION FUNCTION

```

def plot_exploratory_analysis(df_raw, COLORS):
    """ Complete exploratory data analysis visualization """
    # 1. BASIC DATASET ANALYSIS
    fig, axes = plt.subplots(2, 4, figsize=(16, 7))
    fig.suptitle("Dataset Exploratory Analysis", fontsize=14, fontweight='bold', y=1.01)
    axes[0, 0].hist(df_raw['age'].dropna(), bins=30, color=COLORS[0], alpha=0.8, edgecolor='white')
    axes[0, 0].set_title("Age Distribution")
    axes[0, 0].set_xlabel("Age")

    # Click Distribution
    vals = df_raw['click'].value_counts()
    axes[0, 1].bar(['No Click (0)', 'Click (1)'], vals, color=[COLORS[2],
                                                             COLORS[1]], edgecolor='white')
    axes[0, 1].set_title(f"Click Distribution (CTR = {df_raw['click'].mean():.2%})")

    # Categorical Feature Distributions
    cat_cols = ['gender', 'device_type', 'ad_position', 'browsing_history', 'time_of_day']
    for i, col in enumerate(cat_cols):
        ax = axes[(i + 2) // 4][(i + 2) % 4]
        vc = df_raw[col].value_counts()
        ax.barh(vc.index, vc.values, color=COLORS[i % len(COLORS)], alpha=0.8, edgecolor='white')
        ax.set_title(col.replace('_', ' ').title())

    # Hide last empty subplot
    axes[1, 3].axis('off')
    plt.tight_layout()
    plt.show()
    # 2. CTR ANALYSIS

```

```

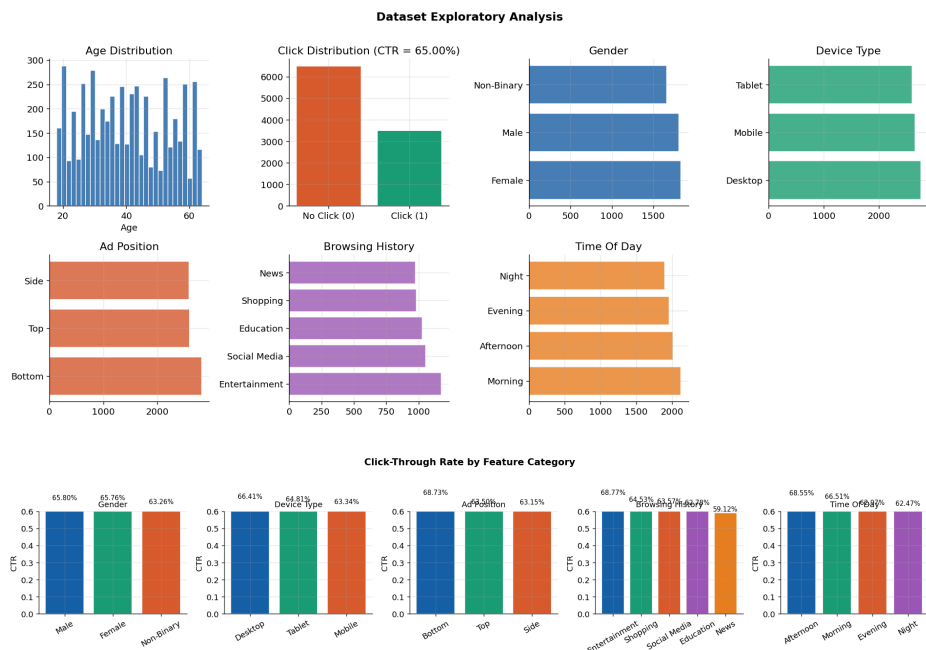
fig, axes = plt.subplots(1, 5, figsize=(18, 4))
fig.suptitle("Click-Through Rate by Feature Category", fontsize=13, fontweight='bold' )

for ax, col in zip( axes,
    ['gender', 'device_type', 'ad_position', 'browsing_history', 'time_of_day'
    ] ):
    ctr = (
        df_raw
        .groupby(col)['click']
        .mean()
        .sort_values(ascending=False)
    )
    bars =
        ax.bar(ctr.index,ctr.values,color=COLORS[:len(ctr)],edgecolor='white')
    ax.set_title( col.replace('_', ' ').title(), fontsize=11 )
    ax.set_ylabel("CTR")
    ax.set_ylim(0, 0.6)
    ax.tick_params( axis='x', rotation=30)
    # Add value labels
    for bar, val in zip(bars, ctr.values):
        ax.text(bar.get_x() + bar.get_width() / 2, val + 0.01,
            f'{val:.2%}', ha='center', fontsize=9)
plt.tight_layout()
plt.show()

```

###EDA Visualization

plot_exploratory_analysis(df_raw, COLORS)



Univariate Statistics: Age Distribution

```

print('=' * 60)
print('Univariate Statistics for Age')
print('=' * 60)
# Calculate statistics for 'age'
age_stats = df_raw['age'].dropna()
print(f"Min Age: {age_stats.min():.0f}")
print(f"Max Age: {age_stats.max():.0f}")
print(f"Median Age: {age_stats.median():.0f}")

```

```

print(f"IQR (Age): {age_stats.quantile(0.75) -
      age_stats.quantile(0.25):.0f}")
print(f"Skewness (Age): {age_stats.skew():.2f}")

=====
Univariate Statistics for Age
=====
Min Age: 18
Max Age: 64
Median Age: 40
IQR (Age): 23
Skewness (Age): 0.10

```

Correlation and Association Analysis

```

def cramers_v(x, y):
    confusion_matrix = pd.crosstab(x, y)
    chi2 = ss.chi2_contingency(confusion_matrix)[0]
    n = confusion_matrix.sum().sum()
    phi2 = chi2 / n
    r, k = confusion_matrix.shape
    phi2corr = max(0, phi2 - ((k-1)*(r-1))/(n-1))
    rcorr = r - ((r-1)**2)/(n-1)
    kcorr = k - ((k-1)**2)/(n-1)
    return np.sqrt(phi2corr / min((kcorr-1), (rcorr-1)))

print('=' * 60)
print('Association with Click Target (Cramer\'s V for Categorical)')
print('=' * 60)
# Assuming CAT_COLS is already defined from previous preprocessing
# For df_raw, we need to re-identify categorical columns relevant before
imputation
cat_cols_raw = ['gender', 'device_type', 'ad_position', 'browsing_history',
                'time_of_day']
associations = {}
for col in cat_cols_raw:
    if col in df_raw.columns:
        # Drop NaNs for Cramer's V calculation to avoid errors
        temp_df = df_raw[[col, 'click']].dropna()
        if not temp_df.empty:
            v_score = cramers_v(temp_df[col], temp_df['click'])
            associations[col] = v_score
        else:
            associations[col] = np.nan # No data to calculate V
association_df = pd.DataFrame.from_dict(associations, orient='index',
                                         columns=['Cramer\'s V'])
association_df = association_df.sort_values(by='Cramer\'s V',
                                           ascending=False)
display(association_df)
print("\nCramer's V measures the strength of association between two nominal
      variables.")
print("Higher values (closer to 1) indicate a stronger association.")

```

```

=====
Association with Click Target (Cramer's V for Categorical)
=====

```

	Cramer's V
browsing_history	0.059580
ad_position	0.051917
time_of_day	0.048858
device_type	0.021169
gender	0.014962

Cramer's V measures the strength of association between two nominal variables.
Higher values (closer to 1) indicate a stronger association.

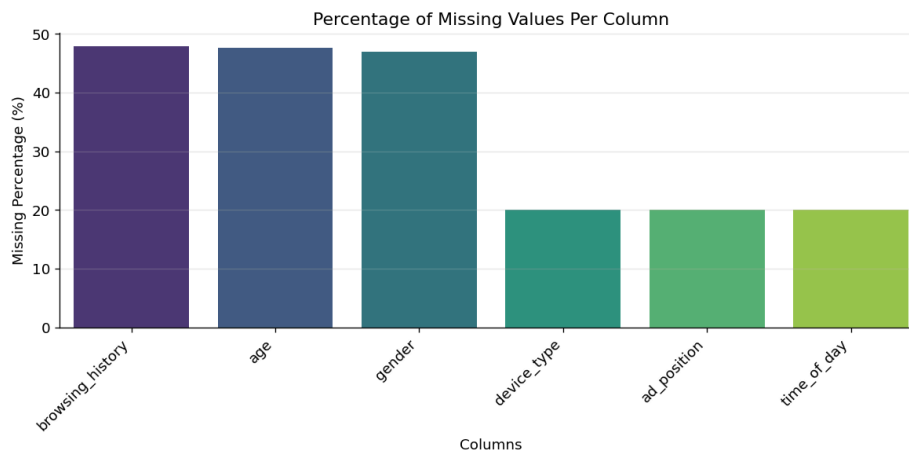
Missing Value Visualization

```
print('=' * 60)
print('Missing Value Visualization')
print('=' * 60)
missing_percent = df_raw.isnull().sum() / len(df_raw) * 100
missing_percent = missing_percent[missing_percent >
                                0].sort_values(ascending=False)
if not missing_percent.empty:
    fig, ax = plt.subplots(figsize=(10, 5))
    sns.barplot(x=missing_percent.index, y=missing_percent.values,
               palette='viridis', ax=ax)
    ax.set_title('Percentage of Missing Values Per Column')
    ax.set_ylabel('Missing Percentage (%)')
    ax.set_xlabel('Columns')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
else:
    print("No missing values to visualize.")
```

=====

Missing Value Visualization

=====



3.Data Preprocessing

Steps:

- 3.1 Drop non-predictive identifiers (id, full_name)
- 3.2 Impute missing values (mean for numerical, mode for categorical)
- 3.3 One-hot encode all categorical features
- 3.4 StandardScale all features
- 3.5 Engineer is_mobile_user feature



Critical constraint: The click column is removed from training data and kept separately for external evaluation only.

```

def preprocess_dataset(df_raw):
    """    Complete preprocessing pipeline:
    1. Separate target      2. Drop identifiers      3. Handle missing values
    4. One-hot encoding     5. Feature engineering  6. Feature scaling
    """

    print("=" * 60)
    print("STARTING PREPROCESSING PIPELINE")
    print("=" * 60)
    # 1. SEPARATE TARGET + DROP IDENTIFIERS
    y_click = df_raw['click'].copy()
    df = df_raw.drop(columns=['id', 'full_name', 'click'])
    print("\n[1] Target Separation Completed")
    print(f"Target Mean CTR      : {y_click.mean():.4f}")
    print(f"Feature Matrix Shape : {df.shape}")
    # 2. DEFINE FEATURE GROUPS
    CAT_COLS =
        ['gender', 'device_type', 'ad_position', 'browsing_history', 'time_of_day']
    NUM_COLS = ['age']
    # 3. HANDLE MISSING VALUES
    num_imputer = SimpleImputer(strategy='mean')
    cat_imputer = SimpleImputer(strategy='most_frequent')
    df[NUM_COLS] = num_imputer.fit_transform(df[NUM_COLS])
    df[CAT_COLS] = cat_imputer.fit_transform(df[CAT_COLS])
    print("\n[2] Missing Value Imputation Completed")
    print("\nRemaining Missing Values:")
    print(df.isnull().sum())
    # 4. ONE-HOT ENCODING
    df_encoded = pd.get_dummies(df, columns=CAT_COLS, drop_first=False)
    print("\n[3] One-Hot Encoding Completed")
    print("Encoded Shape :", df_encoded.shape)
    # 5. FEATURE ENGINEERING
    df_encoded['is_mobile_user'] = (df['device_type'] ==
        'Mobile').astype(int)
    print("\n[4] Feature Engineering Completed")
    print("Added Feature : is_mobile_user")
    print("Updated Shape :", df_encoded.shape)
    # 6. FEATURE SCALING
    scaler = StandardScaler()
    X = scaler.fit_transform(df_encoded)
    print("\n[5] Feature Scaling Completed")
    print(f"Scaled Matrix Shape : {X.shape}")
    print(f"Mean (≈0)           : {X.mean():.4f}")
    print(f"Std (≈1)            : {X.std():.4f}")
    print("\n" + "=" * 60)
    print("PREPROCESSING COMPLETED SUCCESSFULLY")
    print("=" * 60)
    return (X, df, df_encoded, y_click, scaler, CAT_COLS, NUM_COLS)

```

```

X, df, df_encoded, y_click, scaler, CAT_COLS, NUM_COLS =
    preprocess_dataset(df_raw)

```

```

=====
STARTING PREPROCESSING PIPELINE
=====

```

```

[1] Target Separation Completed
Target Mean CTR      : 0.6500
Feature Matrix Shape : (10000, 6)

```

```

[2] Missing Value Imputation Completed

```

```

Remaining Missing Values:
age           0
gender        0
device_type   0

```

```
ad_position          0
browsing_history     0
time_of_day          0
dtype: int64
```

```
[3] One-Hot Encoding Completed
Encoded Shape : (10000, 19)
```

```
[4] Feature Engineering Completed
Added Feature : is_mobile_user
Updated Shape : (10000, 20)
```

```
[5] Feature Scaling Completed
Scaled Matrix Shape : (10000, 20)
Mean (≈0)           : -0.0000
Std (≈1)            : 1.0000
```

```
=====
PREPROCESSING COMPLETED SUCCESSFULLY
=====
```

Justification for is_mobile_user Feature

While device_type is one-hot encoded into device_type_Mobile, device_type_Desktop, and device_type_Tablet, creating an explicit is_mobile_user feature can offer several benefits:

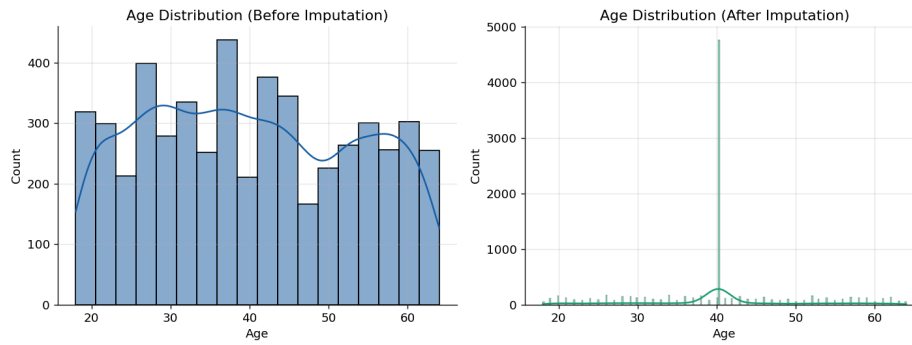
- **Interpretability:** It provides a direct, boolean flag that is easy to interpret as 'is this user on a mobile device?'. This can be more intuitive for business stakeholders or simpler models than interpreting the coefficient of a one-hot encoded column.
- **Model Simplification:** For some models, a single is_mobile_user feature might capture the essence of mobile usage more succinctly, potentially reducing model complexity if mobile behavior is a strong, monolithic predictor.
- **Direct Hypothesis Testing:** It allows for direct testing of hypotheses related specifically to mobile users without needing to combine or interpret multiple device_type one-hot encoded columns.

Pre/Post Imputation Comparison

```
def plot_age_imputation_comparison(df_raw, df, COLORS):
    """Plots the age distribution before and after imputation."""
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    sns.histplot(df_raw['age'].dropna(), kde=True, color=COLORS[0],
                 ax=axes[0])
    axes[0].set_title('Age Distribution (Before Imputation)')
    axes[0].set_xlabel('Age')
    axes[0].set_ylabel('Count')
    sns.histplot(df['age'], kde=True, color=COLORS[1], ax=axes[1])
    axes[1].set_title('Age Distribution (After Imputation)')
    axes[1].set_xlabel('Age')
    axes[1].set_ylabel('Count')
    plt.suptitle('Age Distribution: Before vs. After Imputation',
                 fontsize=14, y=1.02)
    plt.tight_layout(rect=[0, 0, 1, 0.98])
    plt.show()
```

```
plot_age_imputation_comparison(df_raw, df, COLORS)
```

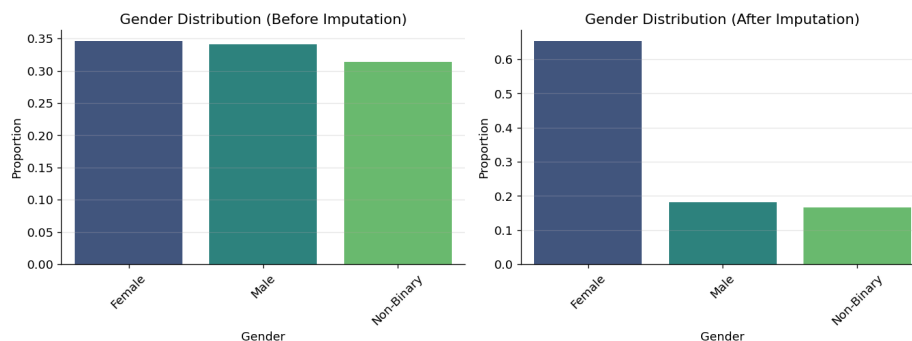

Age Distribution: Before vs. After Imputation



```
def plot_gender_imputation_comparison(df_raw, df, COLORS):
    """Compares 'gender' distribution before and after imputation."""
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    # Before imputation (using df_raw and dropping NaNs for visualization)
    raw_gender_counts = df_raw['gender'].value_counts(normalize=True).sort_index()
    sns.barplot(x=raw_gender_counts.index, y=raw_gender_counts.values,
                palette='viridis', ax=axes[0])
    axes[0].set_title('Gender Distribution (Before Imputation)')
    axes[0].set_xlabel('Gender')
    axes[0].set_ylabel('Proportion')
    axes[0].tick_params(axis='x', rotation=45)
    # After imputation (using df)
    imputed_gender_counts = df['gender'].value_counts(normalize=True).sort_index()
    sns.barplot(x=imputed_gender_counts.index, y=imputed_gender_counts.values, palette='viridis', ax=axes[1])
    axes[1].set_title('Gender Distribution (After Imputation)')
    axes[1].set_xlabel('Gender')
    axes[1].set_ylabel('Proportion')
    axes[1].tick_params(axis='x', rotation=45)
    plt.suptitle('Gender Distribution: Before vs. After Imputation',
                 fontsize=14, y=1.02)
    plt.tight_layout(rect=[0, 0, 1, 0.98])
    plt.show()
```

plot_gender_imputation_comparison(df_raw, df, COLORS)

Gender Distribution: Before vs. After Imputation



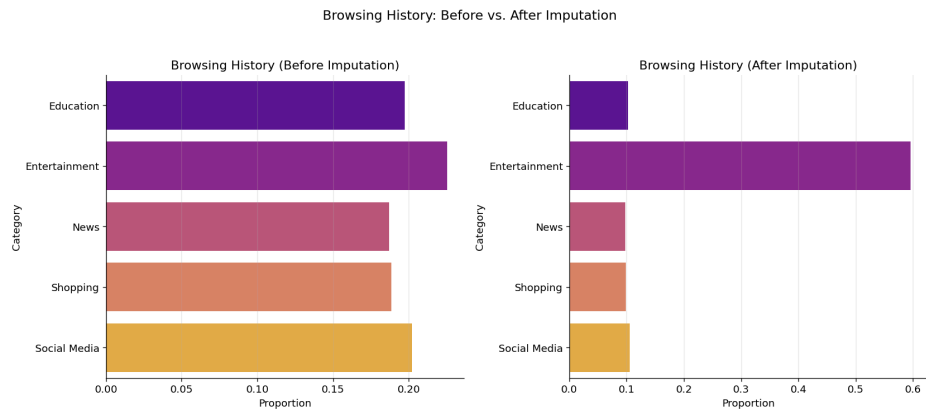
```
def plot_browsing_history_imputation_comparison(df_raw, df, COLORS):
    """Compares 'browsing_history' distribution before and after
    imputation."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    # Before imputation
    raw_bh_counts = df_raw['browsing_history'].value_counts(normalize=True).sort_index()
    sns.barplot(y=raw_bh_counts.index, x=raw_bh_counts.values,
                palette='plasma', ax=axes[0])
```

```

axes[0].set_title('Browsing History (Before Imputation)')
axes[0].set_xlabel('Proportion')
axes[0].set_ylabel('Category')
# After imputation
imputed_bh_counts =
    df['browsing_history'].value_counts(normalize=True).sort_index()
sns.barplot(y=imputed_bh_counts.index, x=imputed_bh_counts.values,
            palette='plasma', ax=axes[1])
axes[1].set_title('Browsing History (After Imputation)')
axes[1].set_xlabel('Proportion')
axes[1].set_ylabel('Category')
plt.suptitle('Browsing History: Before vs. After Imputation',
            fontsize=14, y=1.02)
plt.tight_layout(rect=[0, 0, 1, 0.98])
plt.show()

```

```
plot_browsing_history_imputation_comparison(df_raw, df, COLORS)
```



4. K-Means Clustering

Elbow Method + Silhouette Scores to find Optimal K

```

def find_optimal_k(X, k_range=range(3, 9)):
    """ Find optimal clusters using multiple metrics """
    sse_list = []
    sil_list = []
    db_list = []
    for k in k_range:
        model = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
        labels = model.fit_predict(X)
        sse_list.append(model.inertia_)
        sil_list.append(silhouette_score(X, labels))
        db_list.append(davies_bouldin_score(X, labels))

    optimal_k = list(k_range)[sil_list.index(max(sil_list))]
    print('=' * 60)
    print(f" Optimal K selected: {optimal_k} (Silhouette = {max(sil_list):.4f})")
    print('=' * 60)
    results_k = pd.DataFrame({'K': list(k_range),
                              'SSE': [round(v, 1) for v in sse_list],
                              'Silhouette': [round(v, 4) for v in sil_list],
                              'Davies-Bouldin': [round(v, 4) for v in db_list],
                              })
    print(results_k.to_string(index=False))
    print('=' * 60)
    return optimal_k, sse_list, sil_list, db_list

```

```
OPTIMAL_K, sse_list, sil_list, db_list = find_optimal_k(X)
```

```

=====
Optimal K selected: 5 (Silhouette = 0.1584)
=====

```

K	SSE	Silhouette	Davies-Bouldin
3	159786.8	0.1454	2.4042
4	151367.7	0.1527	2.2387
5	143185.3	0.1584	2.1025
6	137788.5	0.1479	2.1497
7	129994.6	0.1459	1.8985
8	125172.7	0.1317	1.9993

```

=====

```

The optimal number of clusters was selected by considering three factors:

1. Elbow Method – SSE begins to stabilize at K = 4-5.
2. Silhouette Score – Highest value observed at K = 5.
3. Davies-Bouldin Index – Lower values indicate better cluster separation.

From a business perspective, K = 5 provides a good balance between cluster distinctiveness and operational manageability. Creating too many segments increases marketing complexity, while too few segments may hide important user differences.

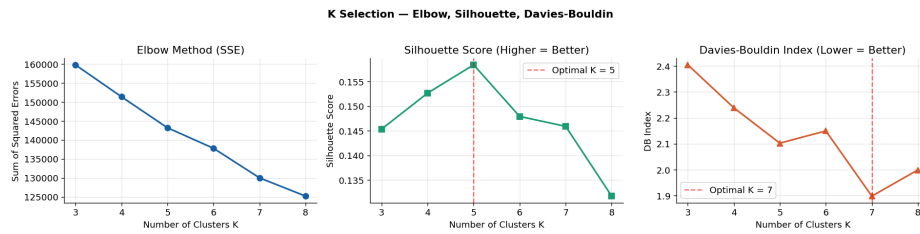
Therefore, K = 5 was selected for both clustering approaches.

```

# PLOT K SELECTION
def plot_k_selection( sse_list,    sil_list,    db_list,    COLORS,
                     k_range=range(3, 9)):
    """    Plot clustering metric comparison    """
    # Safety check
    if len(sil_list) == 0 or len(db_list) == 0:
        print("Metric lists are empty")
        return
    fig, axes = plt.subplots(1, 3, figsize=(16, 4))
    fig.suptitle("K Selection – Elbow, Silhouette, Davies-
        Bouldin", fontsize=13, fontweight='bold' )
    k_vals = list(k_range)
    # 1. Elbow Method
    axes[0].plot(k_vals, sse_list, 'o-', color=COLORS[0], lw=2, markersize=7)
    axes[0].set_title("Elbow Method (SSE)")
    axes[0].set_xlabel("Number of Clusters K")
    axes[0].set_ylabel("Sum of Squared Errors")
    axes[0].set_xticks(k_vals)
    # 2. Silhouette Score
    axes[1].plot(k_vals, sil_list, 's-', color=COLORS[1], lw=2, markersize=7)
    opt_idx = sil_list.index(max(sil_list))
    axes[1].axvline(k_vals[opt_idx], color='red', linestyle='--', alpha=0.6,
        label=f'Optimal K = {k_vals[opt_idx]}')
    axes[1].set_title("Silhouette Score (Higher = Better)")
    axes[1].set_xlabel("Number of Clusters K")
    axes[1].set_ylabel("Silhouette Score")
    axes[1].set_xticks(k_vals)
    axes[1].legend()
    # 3. Davies-Bouldin Index
    axes[2].plot(k_vals, db_list, '^-', color=COLORS[2], lw=2, markersize=7 )
    opt_db = db_list.index(min(db_list))
    axes[2].axvline(
        k_vals[opt_db], color='red', linestyle='--', alpha=0.6, label=f'Optimal K
        = {k_vals[opt_db]}')
    axes[2].set_title("Davies-Bouldin Index (Lower = Better)")
    axes[2].set_xlabel("Number of Clusters K")
    axes[2].set_ylabel("DB Index")
    axes[2].set_xticks(k_vals)
    axes[2].legend()
    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()

```

```
plot_k_selection(sse_list,sil_list, db_list, COLORS )
```



```
# TRAIN KMEANS — using Optimal K and train final model
def train_kmeans(X, n_clusters):
    """ Train KMeans clustering """
    model = KMeans(n_clusters=n_clusters, random_state=RANDOM_STATE,
                    n_init=10 )
    km_labels = model.fit_predict(X)
    silhouette = silhouette_score(X, km_labels)
    db_index = davies_bouldin_score(X, km_labels)
    ch_score = calinski_harabasz_score(X, km_labels)

    print('=' * 60)
    print('KMeans Performance')
    print('=' * 60)
    print('Silhouette Score:', round(silhouette, 4))
    print('Davies-Bouldin Index:', round(db_index, 4))
    print('Calinski-Harabasz Score:', round(ch_score, 4))
    print(f"\nFinal KMeans | " f"Silhouette: {silhouette:.4f} | " f"DB Index: {db_index:.4f} | " f"ch_score: {ch_score:.4f}")
    print(f"Cluster sizes:\n{pd.Series(km_labels).value_counts().sort_index().to_string()}")
    return model, km_labels, silhouette, db_index , ch_score

def plot_silhouette_scores(X, labels, optimal_k, COLORS):
    """Plots the silhouette scores for each sample in each cluster."""
    # Create a subplot with 1 row and 1 column
    fig, ax = plt.subplots(1, 1, figsize=(10, 7))
    ax.set_title("Silhouette plot for K-Means clustering")
    ax.set_xlabel("Silhouette coefficient values")
    ax.set_ylabel("Cluster label")
    # The silhouette coefficient can range from -1, 1 but in this example all
    # and values are positive
    ax.set_xlim([-0.1, 1])
    # The (n_clusters+1)*10 is for inserting blank space between silhouette
    # plots of individual clusters, to make them demarkated clearly.
    ax.set_ylim([0, len(X) + (optimal_k + 1) * 10])

    # Compute the silhouette scores for each sample
    sample_silhouette_values = silhouette_samples(X, labels)
    y_lower = 10
    for i in range(optimal_k):
        # Aggregate the silhouette scores for samples belonging to cluster
        # i, and sort them
        ith_cluster_silhouette_values = \
            sample_silhouette_values[labels == i]

        ith_cluster_silhouette_values.sort()
        size_cluster_i = ith_cluster_silhouette_values.shape[0]
        y_upper = y_lower + size_cluster_i
        color = cm.nipy_spectral(float(i) / optimal_k)
        ax.fill_betweenx(np.arange(y_lower, y_upper), 0,
                        ith_cluster_silhouette_values,
                        facecolor=color, edgecolor=color, alpha=0.7)

        # Label the silhouette plots with their cluster numbers at the middle
        ax.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))
        # Compute the new y_lower for next plot
```

```

y_lower = y_upper + 10 # 10 for the 0 samples

ax.axvline(x=silhouette_score(X, labels), color="red", linestyle="--")
ax.set_yticks([]) # Clear the yaxis labels / ticks
ax.set_xticks([-0.1, 0, 0.2, 0.4, 0.6, 0.8, 1])
plt.show()

### Train kmeans Model

km_model, km_labels, silhouette, db_index , ch_score = train_kmeans(X,
    OPTIMAL_K)
# Aliases used later for ML vs DL comparison
km_sil = silhouette
km_db = db_index

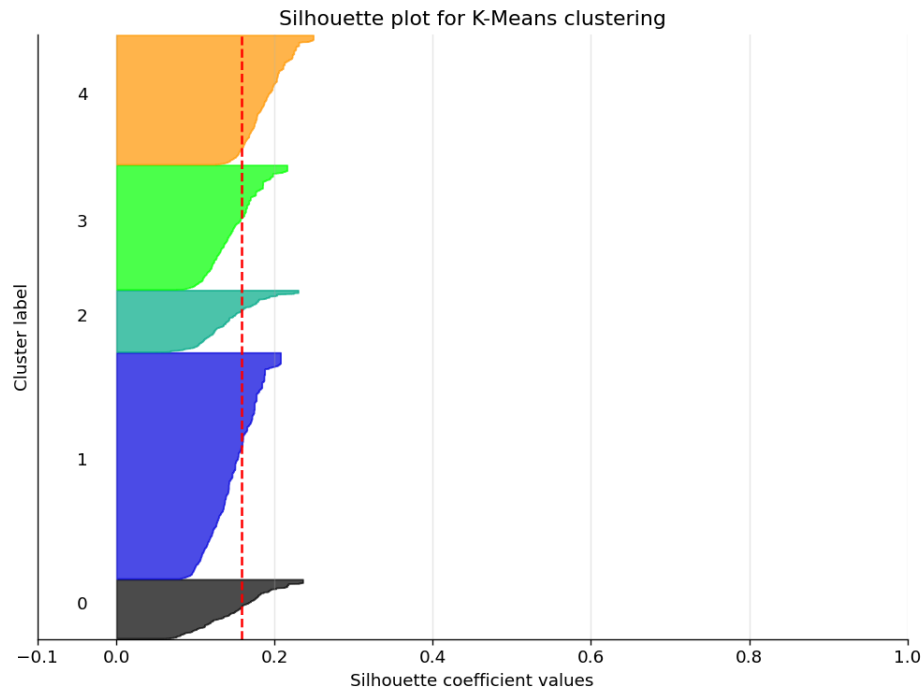
=====
KMeans Performance
=====

Silhouette Score: 0.1584
Davies-Bouldin Index: 2.1025
Calinski-Harabasz Score: 991.4826

Final KMeans | Silhouette: 0.1584 | DB Index: 2.1025 |ch_score: 991.4826
Cluster sizes:
0      984
1     3764
2     1029
3     2071
4     2152

plot_silhouette_scores(X, km_labels, OPTIMAL_K, COLORS)

```



Cluster Size Analysis

The K-Means clustering with $k=5$ resulted in the following cluster distribution:

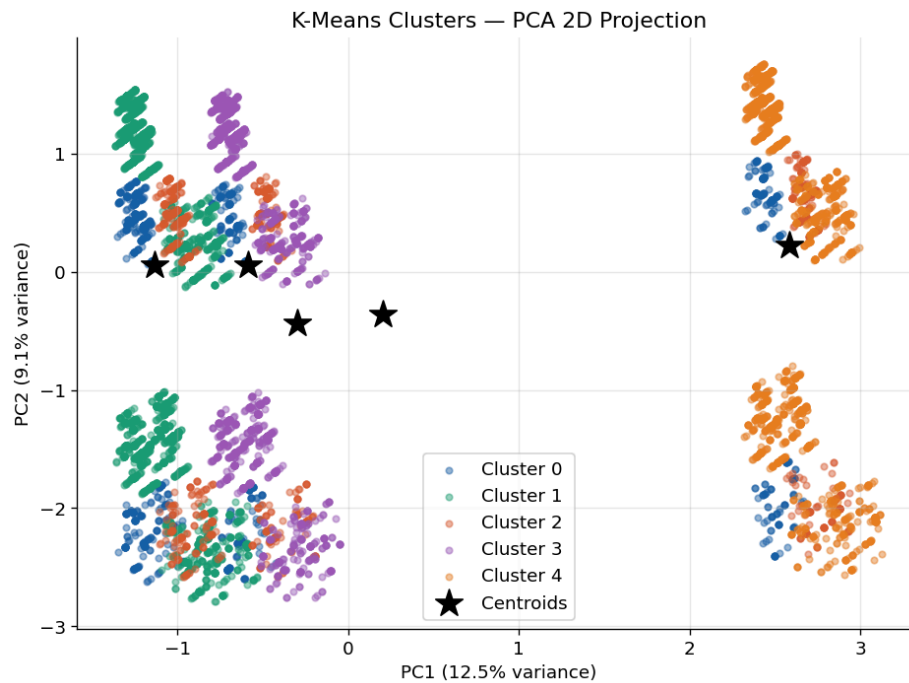
- **Cluster 0:** 984 samples
- **Cluster 1:** 3764 samples
- **Cluster 2:** 1029 samples
- **Cluster 3:** 2071 samples
- **Cluster 4:** 2152 samples

We observe a notable imbalance in cluster sizes, with Cluster 1 being significantly larger than others. This could indicate that some segments are more prevalent, or that the clustering algorithm found a large, somewhat homogeneous group. While not necessarily problematic, significant imbalance can sometimes lead to less distinct profiles for smaller clusters or overly broad generalizations for larger ones. This will be considered during cluster interpretation.

```
# PCA CLUSTER VISUALIZATION FUNCTION
# =====
def plot_pca_kmeans_clusters(X, km_labels, km_model, COLORS):
    """ PCA 2D visualization for KMeans clusters """
    # APPLY PCA
    pca_km = PCA(n_components=2, random_state=42)
    X_km_2d = pca_km.fit_transform(X)
    explained = pca_km.explained_variance_ratio_
    print("=" * 60)
    print("PCA Explained Variance")
    print("=" * 60)
    print(f"PC1 = {explained[0]:.2%} | " f"PC2 = {explained[1]:.2%} | "
          f"Total = {sum(explained):.2%}")
    fig, ax = plt.subplots(figsize=(8, 6))
    unique_clusters = np.unique(km_labels)
    # PLOT EACH CLUSTER
    for k in unique_clusters:
        mask = km_labels == k
        ax.scatter(X_km_2d[mask, 0], X_km_2d[mask,
            1], alpha=0.45, s=15, color=COLORS[k % len(COLORS)],
            label=f'Cluster {k}'
        )
    # PLOT CENTROIDS
    centroids_pca = pca_km.transform(km_model.cluster_centers_)
    ax.scatter(centroids_pca[:, 0], centroids_pca[:,
        1], marker='*', s=300, c='black', zorder=5,
        label='Centroids'
    )
    # LABELS & TITLES
    ax.set_xlabel(f'PC1 ({explained[0]:.1%} variance)')
    ax.set_ylabel(f'PC2 ({explained[1]:.1%} variance)')
    ax.set_title("K-Means Clusters - PCA 2D Projection")
    ax.legend(framealpha=0.8)
    plt.tight_layout()
    plt.show()
    return pca_km, X_km_2d

pca_km, X_km_2d = plot_pca_kmeans_clusters( X, km_labels, km_model,
        COLORS)

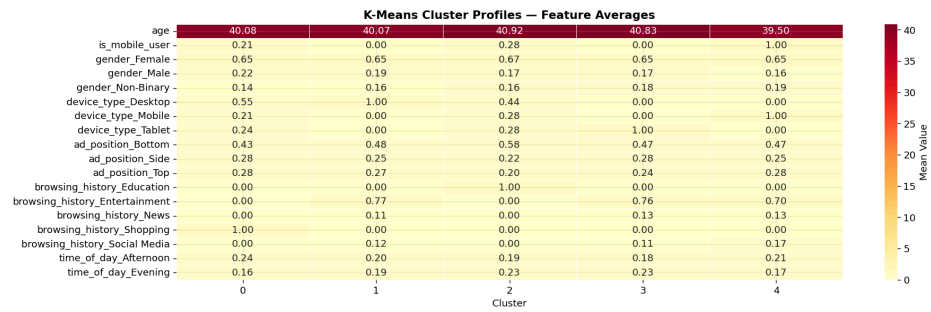
=====
PCA Explained Variance
=====
PC1 = 12.48% | PC2 = 9.12% | Total = 21.60%
```



```
# CLUSTER PROFILING HEATMAP FUNCTION
def plot_cluster_profiles( df_encoded, km_labels, y_click):
    """    Generate cluster profiling heatmap    """
    # CREATE CLUSTER DATAFRAME
    df_km = df_encoded.copy()
    df_km['cluster'] = km_labels
    df_km['click'] = y_click.values
    # COMPUTE CLUSTER PROFILE
    km_profile = df_km.groupby( 'cluster' ).mean()
    print("=" * 60)
    print("Cluster Profile Shape")
    print("=" * 60)
    print(km_profile.shape)
    # SELECT INTERPRETABLE FEATURES
    hmap_cols = [
        c for c in df_km.columns
        if c not in ['cluster', 'click', 'age', 'is_mobile_user']
    ]
    hmap_cols = ['age', 'is_mobile_user' ] + hmap_cols[:16]
    # PLOT HEATMAP
    fig, ax = plt.subplots(figsize=(16, 5))
    sns.heatmap(km_profile[hmap_cols].T, annot=True, fmt='.2f', cmap='YlOrRd', linewidths=0.4, ax=ax,
        cbar_kws={'label': 'Mean Value'} )
    ax.set_title("K-Means Cluster Profiles – Feature Averages", fontsize=13,
        fontweight='bold' )
    ax.set_xlabel("Cluster")
    ax.set_ylabel("")
    plt.tight_layout()
    plt.show()
    return km_profile, hmap_cols

km_profile, hmap_cols = plot_cluster_profiles( df_encoded, km_labels,
    y_click)
```

```
=====
Cluster Profile Shape
=====
(5, 21)
```



```
# CTR ANALYSIS FUNCTION
# =====
def plot_cluster_ctr_analysis(df_encoded,km_labels,y_click,COLORS):
    """    Plot CTR analysis for KMeans clusters    """
    # CREATE DATAFRAME
    df_km = df_encoded.copy()
    df_km['cluster'] = km_labels
    df_km['click'] = y_click.values
    # COMPUTE CTR
    ctr_km = (df_km.groupby('cluster')['click'].mean().sort_index())
    overall_ctr = y_click.mean()
    print("=" * 60)
    print("CTR Analysis")
    print("=" * 60)
    print(f"Overall CTR : {overall_ctr:.4f}")
    # CREATE BAR PLOT
    # =====
    fig, ax = plt.subplots(figsize=(7, 4))
    bars = ax.bar(
        [f'Cluster {k}' for k in
         ctr_km.index],ctr_km.values,color=COLORS[:len(ctr_km)],
        edgecolor='white',          width=0.5 )
    # Overall CTR Line
    ax.axhline(overall_ctr,color='black',linestyle='--
                ',linewidth=1.5,label=f'Overall CTR ({overall_ctr:.2%})')
    ax.set_ylabel("Click-Through Rate (CTR)")
    ax.set_title("K-Means: CTR per Cluster",fontsize=12,fontweight='bold')
    ax.set_ylim(0,max(ctr_km.values) * 1.3)
    # VALUE LABELS
    for bar, val in zip(bars,ctr_km.values):
        ax.text(bar.get_x() + bar.get_width() / 2,val +
                0.005,f'{val:.2%}',ha='center',
                va='bottom',fontsize=11,fontweight='bold' )
    ax.legend()
    plt.tight_layout()
    plt.show()

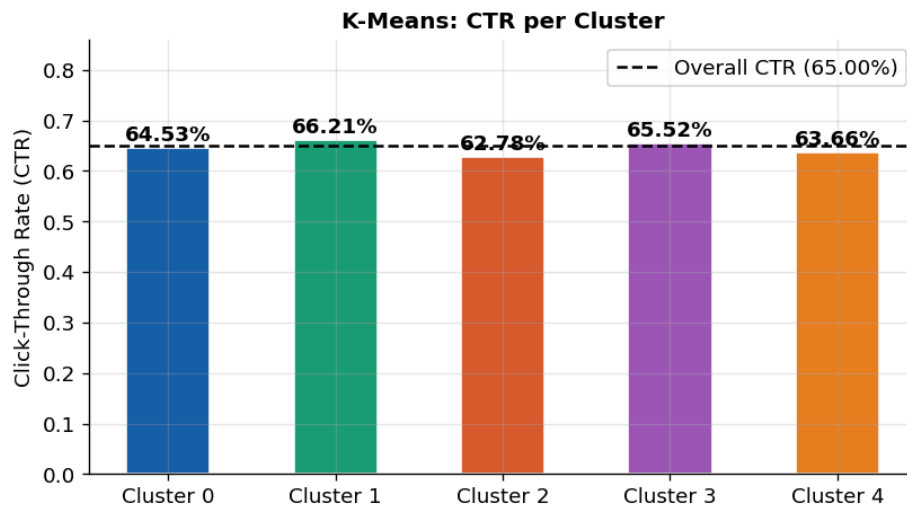
    print("\nKMeans CTR Summary:")
    for k, v in ctr_km.items():
        diff = v - overall_ctr
        print( f"    Cluster {k}: " f"{v:.4f} "
              f"({ '+' if diff >= 0 else '' }{diff:.4f} vs baseline)" )
    print( f"\nCTR Spread (max-min): " f"{ctr_km.max() -
        ctr_km.min():.4f}" )
    return ctr_km, overall_ctr
```

CTR per KMeans Cluster

```
ctr_km, overall_ctr =
    plot_cluster_ctr_analysis(df_encoded,km_labels,y_click,COLORS)
```

```
=====
CTR Analysis
```


=====
Overall CTR : 0.6500



KMeans CTR Summary:

Cluster 0: 0.6453 (-0.0047 vs baseline)
Cluster 1: 0.6621 (+0.0121 vs baseline)
Cluster 2: 0.6278 (-0.0222 vs baseline)
Cluster 3: 0.6552 (+0.0052 vs baseline)
Cluster 4: 0.6366 (-0.0134 vs baseline)

CTR Spread (max-min): 0.0343

5. Autoencoder + K-Means (Deep Learning)

Architecture:

- **Encoder:** Input → Dense(64, ReLU) → Dense(32, ReLU) → Dense(8, ReLU) ←
Latent bottleneck
- **Decoder:** Dense(32, ReLU) → Dense(64, ReLU) → Dense(input_dim, linear)

The encoder is trained to reconstruct the original feature vector through an 8-dimensional bottleneck, forcing it to learn only the most informative user patterns.

```
# BUILD AUTOENCODER FUNCTION
def build_autoencoder(X,latent_dim=8,learning_rate=1e-3,random_state=42):
    """ Build deep autoencoder model """
    # SET RANDOM SEED
    tf.random.set_seed(random_state)
    # INPUT DIMENSION
    input_dim = X.shape[1]
    print("=" * 60)
    print("BUILDING AUTOENCODER")
    print("=" * 60)
    print(f"Input Dimension : {input_dim}")
    print(f"Latent Dimension : {latent_dim}")
    # ENCODER
    inp = keras.Input( shape=(input_dim,), name='input' )
    enc1 = keras.layers.Dense( 64, activation='relu', name='enc1' )(inp)
    enc2 = keras.layers.Dense(32,activation='relu',name='enc2')(enc1)
    latent = keras.layers.Dense(latent_dim,activation='relu',name='latent')
              (enc2)
    # DECODER
    dec1 = keras.layers.Dense(32,activation='relu',name='dec1')(latent)
    dec2 = keras.layers.Dense(64,activation='relu',name='dec2')(dec1)
```

```

output = keras.layers.Dense(input_dim,activation='linear',name='output')
(dec2)
# BUILD MODELS
autoencoder = keras.Model(inp,output,name='Autoencoder')
encoder = keras.Model(inp,latent,name='Encoder')
# COMPILE MODEL
autoencoder.compile(optimizer=keras.optimizers.Adam(learning_rate),loss='mse')
# MODEL SUMMARY
print("\nAutoencoder Summary")
autoencoder.summary()
print("=" * 60)
return autoencoder, encoder

```

```

autoencoder, encoder = build_autoencoder(X,latent_dim=8)
# Latent dimensionality used by the autoencoder bottleneck
LATENT_DIM = 8

```

```

=====
BUILDING AUTOENCODER
=====
Input Dimension : 20
Latent Dimension : 8

```

Autoencoder Summary

Model: "Autoencoder"

Layer (type)	Output Shape
input (InputLayer)	(None, 20)
enc1 (Dense)	(None, 64)
enc2 (Dense)	(None, 32)
latent (Dense)	(None, 8)
dec1 (Dense)	(None, 32)
dec2 (Dense)	(None, 64)
output (Dense)	(None, 20)

Total params: 7,388 (28.86 KB)

Trainable params: 7,388 (28.86 KB)

Non-trainable params: 0 (0.00 B)

```

=====
# TRAIN AUTOENCODER FUNCTION
def train_autoencoder( autoencoder, X, epochs=100,
    batch_size=256,validation_split=0.1, verbose=1):
    """ Train autoencoder model """
    print("=" * 60)
    print("TRAINING AUTOENCODER")
    print("=" * 60)
    print(f"Epochs           : {epochs}")
    print(f"Batch Size          : {batch_size}")
    print(f"Validation Split    : {validation_split}")
    # TRAIN MODEL
    history = autoencoder.fit(X,X,epochs=epochs,batch_size=batch_size,
        validation_split=validation_split,verbose=verbose)

```

```

# FINAL RESULTS
final_train_loss = history.history['loss'][-1]
final_val_loss = history.history['val_loss'][-1]
print("\n" + "=" * 60)
print("TRAINING COMPLETED")
print("=" * 60)
print(f"Final Training Loss : {final_train_loss:.4f}")
print(f"Final Validation Loss : {final_val_loss:.4f}")
return history

```

```

history = train_autoencoder(    autoencoder,    X,    epochs=100,
                                batch_size=256)

```

```

=====
TRAINING AUTOENCODER
=====
Epochs          : 100
Batch Size       : 256
Validation Split : 0.1
Epoch 1/100
36/36 ----- 2s 10ms/step - loss: 0.9181 - val_loss: 0.8167
Epoch 2/100
36/36 ----- 0s 5ms/step - loss: 0.7212 - val_loss: 0.6094
Epoch 3/100
36/36 ----- 0s 5ms/step - loss: 0.5371 - val_loss: 0.4599
Epoch 4/100
36/36 ----- 0s 5ms/step - loss: 0.4189 - val_loss: 0.3687
Epoch 5/100
36/36 ----- 0s 4ms/step - loss: 0.3431 - val_loss: 0.3033
Epoch 6/100
36/36 ----- 0s 5ms/step - loss: 0.2840 - val_loss: 0.2534
Epoch 7/100
36/36 ----- 0s 9ms/step - loss: 0.2375 - val_loss: 0.2122
Epoch 8/100
36/36 ----- 1s 8ms/step - loss: 0.1974 - val_loss: 0.1780
Epoch 9/100
36/36 ----- 0s 7ms/step - loss: 0.1661 - val_loss: 0.1529
Epoch 10/100
36/36 ----- 1s 8ms/step - loss: 0.1439 - val_loss: 0.1357
Epoch 11/100
36/36 ----- 1s 8ms/step - loss: 0.1283 - val_loss: 0.1229
Epoch 12/100
36/36 ----- 0s 5ms/step - loss: 0.1164 - val_loss: 0.1123
Epoch 13/100
36/36 ----- 0s 5ms/step - loss: 0.1069 - val_loss: 0.1039
Epoch 14/100
36/36 ----- 0s 4ms/step - loss: 0.0991 - val_loss: 0.0970
Epoch 15/100
36/36 ----- 0s 5ms/step - loss: 0.0925 - val_loss: 0.0907
Epoch 16/100
36/36 ----- 0s 5ms/step - loss: 0.0869 - val_loss: 0.0853
Epoch 17/100
36/36 ----- 0s 5ms/step - loss: 0.0819 - val_loss: 0.0801
Epoch 18/100
36/36 ----- 0s 5ms/step - loss: 0.0773 - val_loss: 0.0755
Epoch 19/100
36/36 ----- 0s 5ms/step - loss: 0.0730 - val_loss: 0.0713
Epoch 20/100
36/36 ----- 0s 5ms/step - loss: 0.0690 - val_loss: 0.0677
Epoch 21/100
36/36 ----- 0s 5ms/step - loss: 0.0652 - val_loss: 0.0646
Epoch 22/100
36/36 ----- 0s 5ms/step - loss: 0.0618 - val_loss: 0.0616
Epoch 23/100
36/36 ----- 0s 5ms/step - loss: 0.0587 - val_loss: 0.0586
Epoch 24/100
36/36 ----- 0s 5ms/step - loss: 0.0558 - val_loss: 0.0559

```

```
Epoch 25/100
36/36 _____ 0s 5ms/step - loss: 0.0533 - val_loss: 0.0533
Epoch 26/100
36/36 _____ 0s 5ms/step - loss: 0.0509 - val_loss: 0.0511
Epoch 27/100
36/36 _____ 0s 5ms/step - loss: 0.0488 - val_loss: 0.0489
Epoch 28/100
36/36 _____ 0s 4ms/step - loss: 0.0467 - val_loss: 0.0468
Epoch 29/100
36/36 _____ 0s 5ms/step - loss: 0.0449 - val_loss: 0.0449
Epoch 30/100
36/36 _____ 0s 5ms/step - loss: 0.0432 - val_loss: 0.0432
Epoch 31/100
36/36 _____ 0s 6ms/step - loss: 0.0415 - val_loss: 0.0414
Epoch 32/100
36/36 _____ 0s 5ms/step - loss: 0.0400 - val_loss: 0.0399
Epoch 33/100
36/36 _____ 0s 5ms/step - loss: 0.0387 - val_loss: 0.0383
Epoch 34/100
36/36 _____ 0s 5ms/step - loss: 0.0373 - val_loss: 0.0370
Epoch 35/100
36/36 _____ 0s 5ms/step - loss: 0.0361 - val_loss: 0.0361
Epoch 36/100
36/36 _____ 0s 6ms/step - loss: 0.0349 - val_loss: 0.0348
Epoch 37/100
36/36 _____ 0s 5ms/step - loss: 0.0338 - val_loss: 0.0335
Epoch 38/100
36/36 _____ 0s 5ms/step - loss: 0.0327 - val_loss: 0.0324
Epoch 39/100
36/36 _____ 0s 5ms/step - loss: 0.0318 - val_loss: 0.0315
Epoch 40/100
36/36 _____ 0s 5ms/step - loss: 0.0309 - val_loss: 0.0305
Epoch 41/100
36/36 _____ 0s 5ms/step - loss: 0.0301 - val_loss: 0.0297
Epoch 42/100
36/36 _____ 0s 5ms/step - loss: 0.0294 - val_loss: 0.0291
Epoch 43/100
36/36 _____ 0s 5ms/step - loss: 0.0286 - val_loss: 0.0283
Epoch 44/100
36/36 _____ 0s 5ms/step - loss: 0.0280 - val_loss: 0.0277
Epoch 45/100
36/36 _____ 0s 4ms/step - loss: 0.0271 - val_loss: 0.0268
Epoch 46/100
36/36 _____ 0s 5ms/step - loss: 0.0263 - val_loss: 0.0260
Epoch 47/100
36/36 _____ 0s 5ms/step - loss: 0.0256 - val_loss: 0.0254
Epoch 48/100
36/36 _____ 0s 4ms/step - loss: 0.0249 - val_loss: 0.0247
Epoch 49/100
36/36 _____ 0s 5ms/step - loss: 0.0242 - val_loss: 0.0240
Epoch 50/100
36/36 _____ 0s 4ms/step - loss: 0.0237 - val_loss: 0.0233
Epoch 51/100
36/36 _____ 0s 5ms/step - loss: 0.0232 - val_loss: 0.0228
Epoch 52/100
36/36 _____ 0s 5ms/step - loss: 0.0227 - val_loss: 0.0223
Epoch 53/100
36/36 _____ 0s 6ms/step - loss: 0.0222 - val_loss: 0.0219
Epoch 54/100
36/36 _____ 0s 5ms/step - loss: 0.0217 - val_loss: 0.0216
Epoch 55/100
36/36 _____ 0s 4ms/step - loss: 0.0212 - val_loss: 0.0212
Epoch 56/100
36/36 _____ 0s 5ms/step - loss: 0.0207 - val_loss: 0.0209
Epoch 57/100
36/36 _____ 0s 5ms/step - loss: 0.0204 - val_loss: 0.0205
Epoch 58/100
```

36/36 ————— 0s 5ms/step - loss: 0.0199 - val_loss: 0.0201
Epoch 59/100
36/36 ————— 0s 5ms/step - loss: 0.0195 - val_loss: 0.0198
Epoch 60/100
36/36 ————— 0s 5ms/step - loss: 0.0191 - val_loss: 0.0194
Epoch 61/100
36/36 ————— 0s 5ms/step - loss: 0.0187 - val_loss: 0.0191
Epoch 62/100
36/36 ————— 0s 9ms/step - loss: 0.0183 - val_loss: 0.0188
Epoch 63/100
36/36 ————— 0s 7ms/step - loss: 0.0180 - val_loss: 0.0186
Epoch 64/100
36/36 ————— 0s 8ms/step - loss: 0.0176 - val_loss: 0.0182
Epoch 65/100
36/36 ————— 0s 8ms/step - loss: 0.0173 - val_loss: 0.0180
Epoch 66/100
36/36 ————— 1s 9ms/step - loss: 0.0169 - val_loss: 0.0176
Epoch 67/100
36/36 ————— 0s 9ms/step - loss: 0.0166 - val_loss: 0.0174
Epoch 68/100
36/36 ————— 0s 8ms/step - loss: 0.0163 - val_loss: 0.0171
Epoch 69/100
36/36 ————— 1s 5ms/step - loss: 0.0160 - val_loss: 0.0167
Epoch 70/100
36/36 ————— 0s 5ms/step - loss: 0.0157 - val_loss: 0.0164
Epoch 71/100
36/36 ————— 0s 4ms/step - loss: 0.0154 - val_loss: 0.0160
Epoch 72/100
36/36 ————— 0s 4ms/step - loss: 0.0151 - val_loss: 0.0157
Epoch 73/100
36/36 ————— 0s 5ms/step - loss: 0.0148 - val_loss: 0.0153
Epoch 74/100
36/36 ————— 0s 4ms/step - loss: 0.0145 - val_loss: 0.0150
Epoch 75/100
36/36 ————— 0s 5ms/step - loss: 0.0142 - val_loss: 0.0147
Epoch 76/100
36/36 ————— 0s 4ms/step - loss: 0.0140 - val_loss: 0.0145
Epoch 77/100
36/36 ————— 0s 4ms/step - loss: 0.0138 - val_loss: 0.0142
Epoch 78/100
36/36 ————— 0s 5ms/step - loss: 0.0135 - val_loss: 0.0140
Epoch 79/100
36/36 ————— 0s 5ms/step - loss: 0.0133 - val_loss: 0.0138
Epoch 80/100
36/36 ————— 0s 5ms/step - loss: 0.0131 - val_loss: 0.0136
Epoch 81/100
36/36 ————— 0s 5ms/step - loss: 0.0129 - val_loss: 0.0134
Epoch 82/100
36/36 ————— 0s 4ms/step - loss: 0.0127 - val_loss: 0.0131
Epoch 83/100
36/36 ————— 0s 4ms/step - loss: 0.0125 - val_loss: 0.0129
Epoch 84/100
36/36 ————— 0s 6ms/step - loss: 0.0123 - val_loss: 0.0128
Epoch 85/100
36/36 ————— 0s 5ms/step - loss: 0.0121 - val_loss: 0.0126
Epoch 86/100
36/36 ————— 0s 5ms/step - loss: 0.0119 - val_loss: 0.0125
Epoch 87/100
36/36 ————— 0s 5ms/step - loss: 0.0117 - val_loss: 0.0123
Epoch 88/100
36/36 ————— 0s 4ms/step - loss: 0.0115 - val_loss: 0.0121
Epoch 89/100
36/36 ————— 0s 5ms/step - loss: 0.0114 - val_loss: 0.0120
Epoch 90/100
36/36 ————— 0s 5ms/step - loss: 0.0112 - val_loss: 0.0119
Epoch 91/100
36/36 ————— 0s 5ms/step - loss: 0.0110 - val_loss: 0.0117

```

Epoch 92/100
36/36 _____ 0s 4ms/step - loss: 0.0109 - val_loss: 0.0115
Epoch 93/100
36/36 _____ 0s 5ms/step - loss: 0.0107 - val_loss: 0.0114
Epoch 94/100
36/36 _____ 0s 5ms/step - loss: 0.0106 - val_loss: 0.0113
Epoch 95/100
36/36 _____ 0s 4ms/step - loss: 0.0104 - val_loss: 0.0112
Epoch 96/100
36/36 _____ 0s 5ms/step - loss: 0.0103 - val_loss: 0.0111
Epoch 97/100
36/36 _____ 0s 4ms/step - loss: 0.0102 - val_loss: 0.0111
Epoch 98/100
36/36 _____ 0s 4ms/step - loss: 0.0101 - val_loss: 0.0110
Epoch 99/100
36/36 _____ 0s 5ms/step - loss: 0.0099 - val_loss: 0.0109
Epoch 100/100
36/36 _____ 0s 4ms/step - loss: 0.0098 - val_loss: 0.0108

```

```

=====
TRAINING COMPLETED
=====

```

```

Final Training Loss   : 0.0098
Final Validation Loss : 0.0108

```

```

# PLOT AUTOENCODER TRAINING CURVES

```

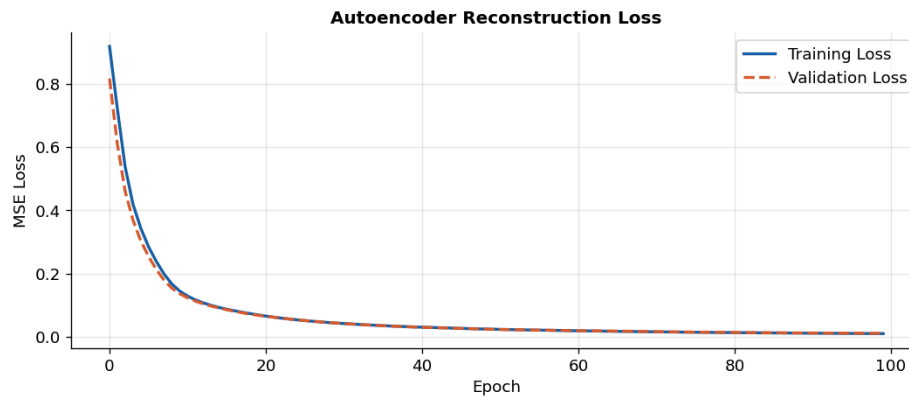
```

def plot_training_curves(history, COLORS):
    """    Plot autoencoder training and validation loss    """

    # EXTRACT LOSSES
    train_loss = history.history['loss']
    val_loss = history.history['val_loss']
    fig, ax = plt.subplots(figsize=(9, 4))
    # Training Loss
    ax.plot(train_loss, label='Training Loss', color=COLORS[0], linewidth=2)
    # Validation Loss
    ax.plot(val_loss, label='Validation Loss', color=COLORS[2], linewidth=2, linestyle='--')
    # LABELS & TITLES
    ax.set_xlabel("Epoch")
    ax.set_ylabel("MSE Loss")
    ax.set_title("Autoencoder Reconstruction Loss", fontsize=12, fontweight='bold')
    ax.legend()
    plt.tight_layout()
    plt.show()
    # FINAL LOSS VALUES
    print("=" * 60)
    print("Training Summary")
    print("=" * 60)
    print(f"Final Training Loss   : {train_loss[-1]:.4f}")
    print(f"Final Validation Loss : {val_loss[-1]:.4f}")

plot_training_curves(history, COLORS)

```



```
=====
Training Summary
=====
Final Training Loss   : 0.0098
Final Validation Loss : 0.0108

# EXTRACT LATENT FEATURES FUNCTION
def extract_latent_features(    encoder,    X):
    """    Extract Latent embeddings from encoder    """
    print("=" * 60)
    print("EXTRACTING LATENT FEATURES")
    print("=" * 60)
    X_latent = encoder.predict(X,verbose=0) # GENERATE LATENT
    REPRESENTATIONS
    # DISPLAY STATISTICS
    print(f"Latent Space Shape : {X_latent.shape} (users x latent dims) ")
    )
    print(f"Latent Mean      : {X_latent.mean():.4f}")
    )
    print(f"Latent Std       : {X_latent.std():.4f}")
    )
    print("=" * 60)
    return X_latent

X_latent = extract_latent_features(    encoder,    X)
```

```
=====
EXTRACTING LATENT FEATURES
=====
Latent Space Shape : (10000, 8) (users x latent dims)
Latent Mean      : 7.9905
Latent Std       : 4.5499
=====
```

The Autoencoder compresses the original high-dimensional user data into an 8-dimensional latent representation.

Unlike raw features, latent features capture hidden relationships between demographic, device, and browsing behaviors.

As a result, users with similar engagement patterns are grouped together more effectively, leading to more compact and meaningful audience segments.

Reconstruction Error Analysis

To validate the quality of the autoencoder, we'll examine the reconstruction error. A low reconstruction error indicates that the autoencoder is effectively capturing the important features of the input data. We'll visualize the distribution of these errors to understand how well each sample is reconstructed.

```
def plot_reconstruction_errors(autoencoder, X, COLORS, feature_names):
    # Get reconstructed data from the autoencoder
```

```

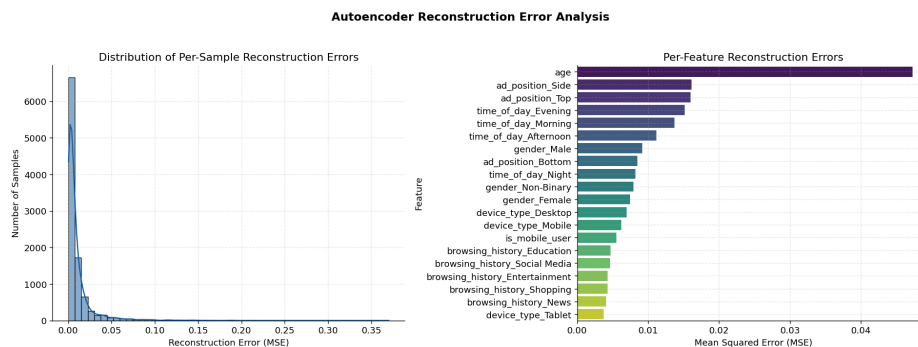
X_reconstructed = autoencoder.predict(X, verbose=0)
# Calculate the Mean Squared Error (MSE) for each sample (row-wise
  average of squared differences)
per_sample_reconstruction_errors = np.mean(np.square(X -
  X_reconstructed), axis=1)
# Calculate the Mean Squared Error (MSE) for each feature (column-wise
  average of squared differences)
per_feature_reconstruction_errors = np.mean(np.square(X -
  X_reconstructed), axis=0)
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Autoencoder Reconstruction Error Analysis', fontsize=14,
  fontweight='bold')

# Plot 1: Distribution of Per-Sample Reconstruction Errors
sns.histplot(per_sample_reconstruction_errors, bins=50, kde=True,
  color=COLORS[0], ax=axes[0])
axes[0].set_title('Distribution of Per-Sample Reconstruction Errors')
axes[0].set_xlabel('Reconstruction Error (MSE)')
axes[0].set_ylabel('Number of Samples')
apply_plot_settings(axes[0], COLORS) # Apply custom settings

# Plot 2: Per-Feature Reconstruction Errors
# Create a DataFrame for easier plotting with seaborn
feature_error_df = pd.DataFrame({
  'Feature': feature_names, 'MSE': per_feature_reconstruction_errors
}).sort_values(by='MSE', ascending=False)
sns.barplot(x='MSE', y='Feature', data=feature_error_df,
  palette='viridis', ax=axes[1])
axes[1].set_title('Per-Feature Reconstruction Errors')
axes[1].set_xlabel('Mean Squared Error (MSE)')
axes[1].set_ylabel('Feature')
apply_plot_settings(axes[1], COLORS) # Apply custom settings
plt.tight_layout(rect=[0, 0, 1, 0.95]) # Adjust layout to prevent
  subtitle overlap
plt.show()
print(f"Overall Average Reconstruction Error (per-sample):
  {np.mean(per_sample_reconstruction_errors):.4f}")
print(f"Max Per-Sample Reconstruction Error:
  {np.max(per_sample_reconstruction_errors):.4f}")
print(f"\nFeatures with highest reconstruction error:")
print(feature_error_df.head())

```

plot_reconstruction_errors(autoencoder,X,COLORS, df_encoded.columns)



Overall Average Reconstruction Error (per-sample): 0.0103

Max Per-Sample Reconstruction Error: 0.3699

Features with highest reconstruction error:

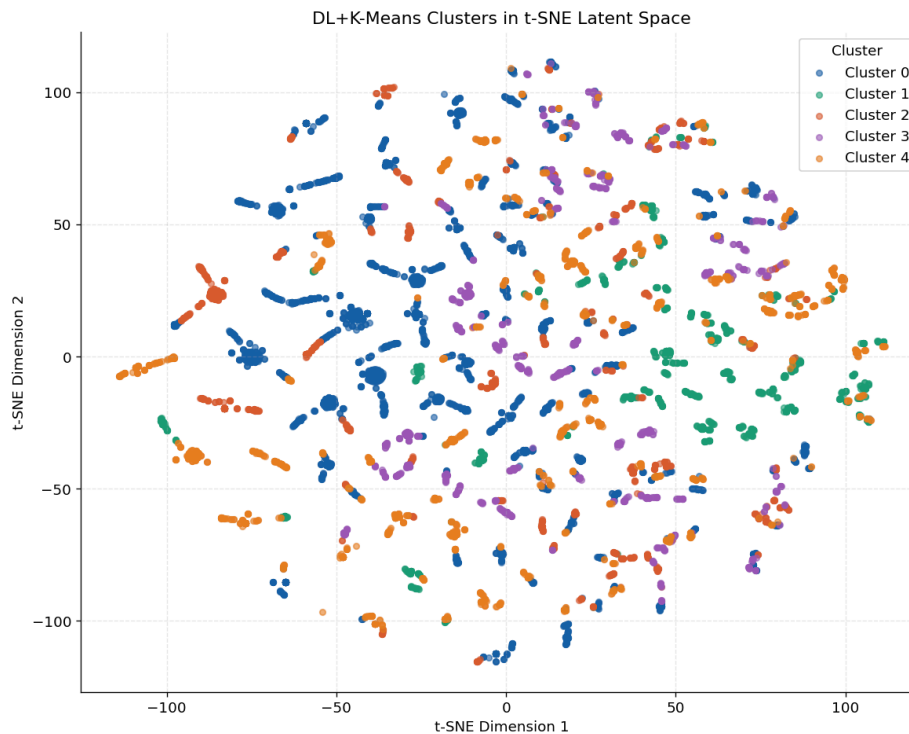
	Feature	MSE
0	age	0.047270
8	ad_position_Side	0.016105
9	ad_position_Top	0.015977
16	time_of_day_Evening	0.015140
17	time_of_day_Morning	0.013719

Latent Space Visualization (t-SNE)

While PCA provides a linear dimensionality reduction, t-SNE (t-Distributed Stochastic Neighbor Embedding) is a non-linear technique that is often more effective at visualizing high-dimensional data by mapping it to a 2D or 3D space, preserving local structures. This can provide a clearer picture of how clusters are formed in the latent space learned by the autoencoder.

```
def plot_latent_tsne(X_latent, dl_labels, COLORS, RANDOM_STATE):
    """Applies t-SNE to latent features and plots the clusters."""
    # Apply t-SNE to the latent features
    tsne = TSNE(n_components=2, random_state=RANDOM_STATE, perplexity=30,
                n_iter=1000)
    X_latent_tsne = tsne.fit_transform(X_latent)
    # Plot the t-SNE transformed latent space, colored by DL+KMeans clusters
    fig, ax = plt.subplots(figsize=(10, 8))
    for k in np.unique(dl_labels):
        mask = dl_labels == k
        ax.scatter(X_latent_tsne[mask, 0], X_latent_tsne[mask, 1],
                  alpha=0.6, s=20, color=COLORS[k % len(COLORS)],
                  label=f'Cluster {k}')
    ax.set_title('DL+K-Means Clusters in t-SNE Latent Space')
    ax.set_xlabel('t-SNE Dimension 1')
    ax.set_ylabel('t-SNE Dimension 2')
    ax.legend(title='Cluster')
    apply_plot_settings(ax, COLORS) # Apply consistent plot settings
    plt.tight_layout()
    plt.show()
    return X_latent_tsne

# Call the function to plot the t-SNE visualization
X_latent_tsne_result = plot_latent_tsne(X_latent, dl_labels, COLORS,
                                       RANDOM_STATE)
```



```
def find_optimal_k_latent(X_latent, K_RANGE, COLORS):
    """ Find optimal K for latent-space clustering using Silhouette and
        Davies-Bouldin metrics """
    sil_dl_list = []
    db_dl_list = []
    ch_dl_list = [] # Add ch_dl_list
```

```

k_vals = list(K_RANGE)
print("=" * 60)
print("FINDING OPTIMAL K IN LATENT SPACE")
print("=" * 60)

for k in K_RANGE:
    km_dl = KMeans(n_clusters=k,random_state=42,n_init=10)
    labels = km_dl.fit_predict(X_latent)
    sil_score = silhouette_score(X_latent,labels)
    db_score = davies_bouldin_score(X_latent,labels)
    ch_score = calinski_harabasz_score(X_latent,labels) # Calculate CH
    score

    sil_dl_list.append(sil_score)
    db_dl_list.append(db_score)
    ch_dl_list.append(ch_score) # Append CH score

    print(f"K = {k} | " f"Silhouette = {sil_score:.4f} | "
          f"DB Index = {db_score:.4f} | " f"CH Score = {ch_score:.4f}"
        )
# FIND OPTIMAL K - Silhouette is used to determine optimal K
optimal_k_dl = k_vals[ sil_dl_list.index( max(sil_dl_list) ) ]
fig, axes = plt.subplots(1,3,figsize=(16, 4)) # Adjust subplot layout for
3 plots
# Silhouette Plot
axes[0].plot(k_vals,sil_dl_list,'o-
',color=COLORS[1],linewidth=2,markersize=7)
axes[0].axvline(optimal_k_dl,color='red',linestyle='--',alpha=0.6)
axes[0].set_title("DL Latent Space – Silhouette Score")
axes[0].set_xlabel("K")
axes[0].set_ylabel("Silhouette Score")
axes[0].set_xticks(k_vals)

# DB Index Plot
axes[1].plot(k_vals,db_dl_list,'s-
',color=COLORS[2],linewidth=2,markersize=7)
axes[1].axvline(k_vals[db_dl_list.index(min(db_dl_list))],color='red',linestyle='--
',alpha=0.6)
axes[1].set_title("DL Latent Space – DB Index")
axes[1].set_xlabel("K")
axes[1].set_ylabel("Davies-Bouldin Index")
axes[1].set_xticks(k_vals)

# CH Score Plot (New subplot)
axes[2].plot(k_vals,ch_dl_list,'^-',color=COLORS[3], # Use a different
color
linewidth=2,markersize=7 )
axes[2].axvline(k_vals[ch_dl_list.index(max(ch_dl_list))], # Max is
better for CH
color='red',linestyle='--',alpha=0.6 )
axes[2].set_title("DL Latent Space – Calinski-Harabasz Score")
axes[2].set_xlabel("K")
axes[2].set_ylabel("CH Score")
axes[2].set_xticks(k_vals)

plt.tight_layout()
plt.show()
print("\n" + "=" * 60)
print(f"Optimal K (DL) : "f""f"{optimal_k_dl}")
print("=" * 60)
return (optimal_k_dl,sil_dl_list,db_dl_list, ch_dl_list)

OPTIMAL_K_DL, sil_dl_list, db_dl_list, ch_dl_list = find_optimal_k_latent(
X_latent, range(3, 9), COLORS )

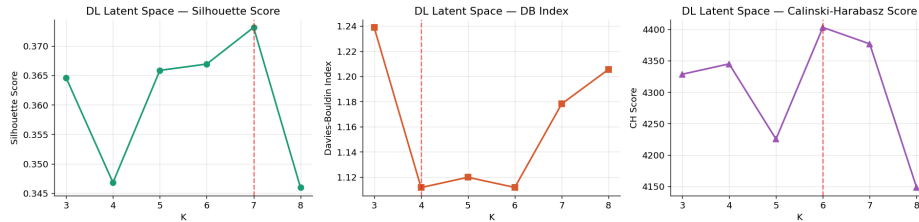
```

FINDING OPTIMAL K IN LATENT SPACE

```

K = 3 | Silhouette = 0.3646 | DB Index = 1.2389 | CH Score = 4328.3911
K = 4 | Silhouette = 0.3468 | DB Index = 1.1116 | CH Score = 4344.8770
K = 5 | Silhouette = 0.3658 | DB Index = 1.1198 | CH Score = 4225.2368
K = 6 | Silhouette = 0.3669 | DB Index = 1.1117 | CH Score = 4402.8652
K = 7 | Silhouette = 0.3731 | DB Index = 1.1781 | CH Score = 4376.6343
K = 8 | Silhouette = 0.3460 | DB Index = 1.2056 | CH Score = 4148.2051

```



Optimal K (DL) : 7

```

# — Final DL K-Means model —————
km_dl_final = KMeans(n_clusters=OPTIMAL_K_DL, random_state=42, n_init=10)
dl_labels = km_dl_final.fit_predict(X_latent)
dl_sil = silhouette_score(X_latent, dl_labels)
dl_db = davies_bouldin_score(X_latent, dl_labels)
dl_ch = calinski_harabasz_score(X_latent, dl_labels) # Calculate dl_ch

print(f"DL+KMeans | Silhouette: {dl_sil:.4f} | DB Index: {dl_db:.4f} | CH
      Score: {dl_ch:.4f}")
print(f"Cluster
      sizes:\n{pd.Series(dl_labels).value_counts().sort_index().to_string()}")

```

DL+KMeans | Silhouette: 0.3731 | DB Index: 1.1781 | CH Score: 4376.6343

Cluster sizes:

```

0    961
1    1929
2    1571
3     984
4     976
5    2454
6    1125

```

```

# — PCA 2D Visualization of Latent Clusters —————
def plot_pca_cluster_comparison(X_latent, X_km_2d, km_labels, dl_labels, pca_km, colors, random_state=42,)
    """Plots a 2D PCA comparison between raw and latent space clustering."""
    # Process Latent space PCA
    pca_dl = PCA(n_components=2, random_state=random_state)
    X_dl_2d = pca_dl.fit_transform(X_latent)
    exp_dl = pca_dl.explained_variance_ratio_
    # Setup plot grid
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    titles = ["K-Means (raw features)", "DL+K-Means (latent features)"]
    datasets = [
        (X_km_2d, km_labels, pca_km.explained_variance_ratio_),
        (X_dl_2d, dl_labels, exp_dl),
    ]
    # Iterate and populate subplots
    for ax, (pts, lbls, ev), title in zip(axes, datasets, titles):
        for k in np.unique(lbls):
            mask = lbls == k
            ax.scatter(pts[mask, 0], pts[mask, 1], alpha=0.4, s=12, color=colors[k % len(colors)], label=f"Cluster
            {k}", )

```

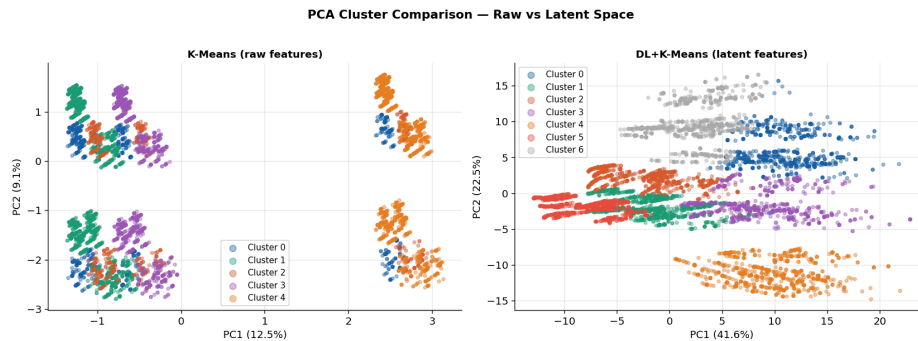
```

ax.set_xlabel(f"PC1 ({ev[0]:.1%})")
ax.set_ylabel(f"PC2 ({ev[1]:.1%})")
ax.set_title(title, fontsize=11, fontweight="bold")
ax.legend(markerscale=2, fontsize=9)

plt.suptitle("PCA Cluster Comparison – Raw vs Latent
             Space", fontsize=13, fontweight="bold", y=1.02, )
plt.tight_layout()
return fig, axes

fig, axes =
    plot_pca_cluster_comparison(X_latent=X_latent, X_km_2d=X_km_2d, km_labels=km_labels,
                               dl_labels=dl_labels, pca_km=pca_km, colors=COLORS,)
plt.show()

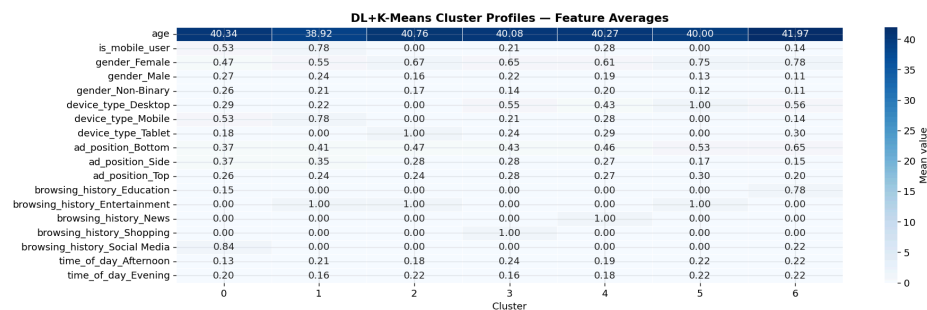
```



```

# — DL Cluster Profiles —————
df_dl = df_encoded.copy()
df_dl['cluster'] = dl_labels
df_dl['click'] = y_click.values
dl_profile = df_dl.groupby('cluster').mean()
fig, ax = plt.subplots(figsize=(16, 5))
sns.heatmap(dl_profile[hmap_cols].T, annot=True, fmt='.2f', cmap='Blues',
            linewidths=0.4, ax=ax, cbar_kws={'label': 'Mean value'})
ax.set_title("DL+K-Means Cluster Profiles – Feature Averages", fontsize=13,
             fontweight='bold')
ax.set_xlabel("Cluster")
ax.set_ylabel("")
plt.tight_layout()
plt.show()

```



```

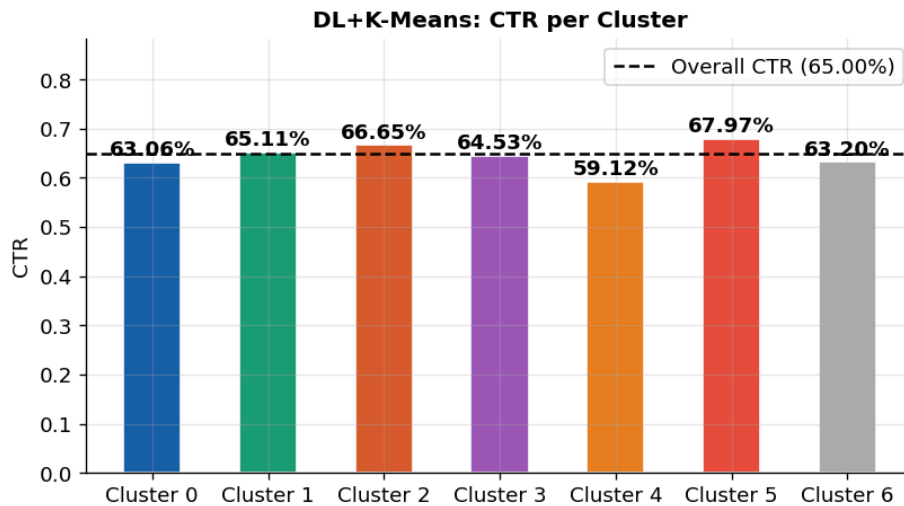
# — CTR per DL Cluster —————
ctr_dl = df_dl.groupby('cluster')['click'].mean().sort_index()
fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(
    [f'Cluster {k}' for k in ctr_dl.index], ctr_dl.values,
    color=COLORS[:OPTIMAL_K_DL], edgecolor='white', width=0.5
)
ax.axhline(overall_ctr, color='black', ls='--', lw=1.5, label=f'Overall CTR
({overall_ctr:.2%})')
ax.set_ylabel("CTR")
ax.set_title("DL+K-Means: CTR per Cluster", fontsize=12, fontweight='bold')
ax.set_ylim(0, max(ctr_dl.values) * 1.3)

```

```

for bar, val in zip(bars, ctr_dl.values):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.005, f'{val:.2%}',
            ha='center', va='bottom', fontsize=11, fontweight='bold')
ax.legend()
plt.tight_layout()
plt.show()

```



6. Model Comparison: ML vs DL

```

comparison = pd.DataFrame({
    'Metric': ['Silhouette Score ↑', 'Davies-Bouldin Index ↓', 'Calinski-
        Harabasz Score ↑',
        'Max Cluster CTR', 'Min Cluster CTR', 'CTR Spread (max - min)', 'Optimal
        K', ],
    'KMeans (ML)': [ round(km_sil, 4), round(km_db, 4), round(ch_score, 4),
        f'{ctr_km.max():.2%}', f'{ctr_km.min():.2%}', f'{(ctr_km.max() -
        ctr_km.min()):.4f}',
        OPTIMAL_K,
    ],
    'Autoencoder+KMeans (DL)': [
        round(dl_sil, 4), round(dl_db, 4), round(dl_ch, 4),
        f'{ctr_dl.max():.2%}', f'{ctr_dl.min():.2%}', f'{(ctr_dl.max() -
        ctr_dl.min()):.4f}',
        OPTIMAL_K_DL, ], })
print(comparison.to_string(index=False))

```

	Metric	KMeans (ML)	Autoencoder+KMeans (DL)
	Silhouette Score ↑	0.1584	0.3731
	Davies-Bouldin Index ↓	2.1025	1.1781
	Calinski-Harabasz Score ↑	991.4826	4376.634277
	Max Cluster CTR	66.21%	67.97%
	Min Cluster CTR	62.78%	59.12%
	CTR Spread (max - min)	0.0343	0.0885
	Optimal K	5	7

```

def plot_ml_vs_dl_comparison(km_sil, dl_sil, km_db, dl_db, km_ch,
    dl_ch, ctr_km, ctr_dl, overall_ctr, COLORS):
    """ ML vs DL clustering comparison visualization """
    # Create Figure - change to 2x2 subplots
    fig, axes = plt.subplots(2, 2, figsize=(15, 10)) # Changed to 2 rows, 2
        columns
    axes = axes.flatten() # Flatten the 2x2 array of axes for easy iteration
    fig.suptitle("ML vs DL Clustering - Head-to-Head
        Comparison", fontsize=13, fontweight='bold')

    labels = ['KMeans (ML)', 'DL+KMeans']

```

```

x = np.arange(len(labels))

# 1. Silhouette Score
axes[0].bar(x,[km_sil, dl_sil],color=[COLORS[0],
    COLORS[1]],width=0.4,edgecolor='white' )
axes[0].set_title("Silhouette Score (↑ better)")
axes[0].set_xticks(x)
axes[0].set_xticklabels(labels)
for xi, value in zip(x, [km_sil, dl_sil]):
    axes[0].text(xi,value +
        0.003,f'{value:.4f}',ha='center',fontweight='bold' )

# 2. Davies-Bouldin Index
axes[1].bar(x,[km_db, dl_db],color=[COLORS[0],
    COLORS[1]],width=0.4,edgecolor='white' )
axes[1].set_title("DB Index (↓ better)")
axes[1].set_xticks(x)
axes[1].set_xticklabels(labels)

for xi, value in zip(x, [km_db, dl_db]):
    axes[1].text(xi,value +
        0.02,f'{value:.4f}',ha='center',fontweight='bold')

# 3. Calinski-Harabasz Score (New Plot)
axes[2].bar(x,[km_ch, dl_ch],color=[COLORS[0],
    COLORS[1]],width=0.4,edgecolor='white' )

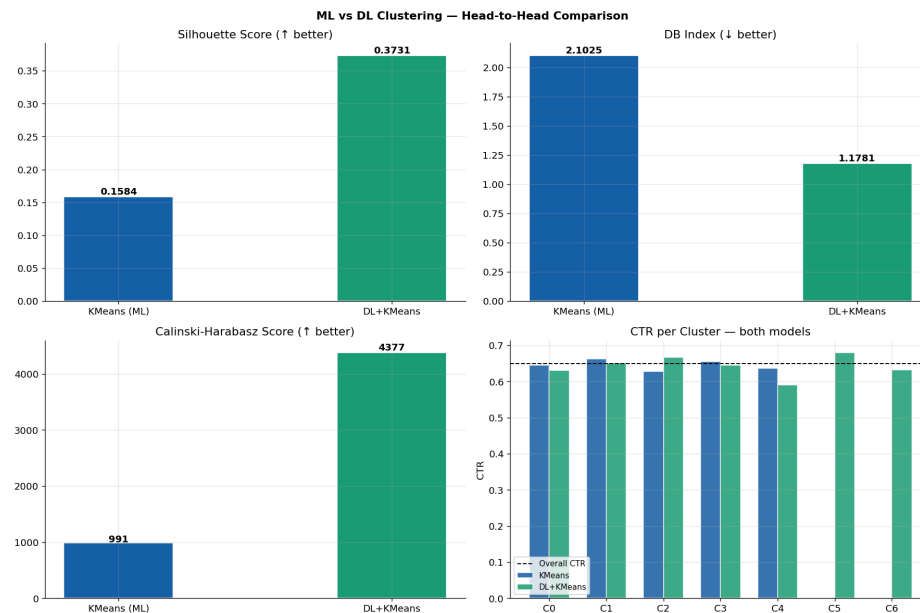
axes[2].set_title("Calinski-Harabasz Score (↑ better)")
axes[2].set_xticks(x)
axes[2].set_xticklabels(labels)

for xi, value in zip(x, [km_ch, dl_ch]):
    axes[2].text( xi,value + (value *
        0.01),f'{value:.0f}',ha='center',fontweight='bold' )

# 4. CTR per Cluster (moved to axes[3])
km_vals = list(ctr_km.values)
dl_vals = list(ctr_dl.values)
# Handle unequal cluster counts
max_clusters = max(len(km_vals), len(dl_vals))
# Pad smaller list with NaN
km_vals += [np.nan] * (max_clusters - len(km_vals))
dl_vals += [np.nan] * (max_clusters - len(dl_vals))
cluster_labels = [f'C{k}' for k in range(max_clusters)]
bw = 0.35
xi = np.arange(max_clusters)
# ML bars
axes[3].bar(xi - bw/2,km_vals,width=bw,label='KMeans',color=COLORS[0],
    alpha=0.85, edgecolor='white' )
# DL bars
axes[3].bar(xi +
    bw/2,dl_vals,width=bw,label='DL+KMeans',color=COLORS[1],alpha=0.85,edgecolor='white'
)
# Overall CTR Line
axes[3].axhline(overall_ctr,linestyle='--
    ',color='black',linewidth=1.2,label='Overall CTR')
axes[3].set_xticks(xi)
axes[3].set_xticklabels(cluster_labels)
axes[3].set_title("CTR per Cluster – both models")
axes[3].set_ylabel("CTR")
axes[3].legend(fontsize=9)
plt.tight_layout()
plt.show()

plot_ml_vs_dl_comparison(km_sil=km_sil,dl_sil=dl_sil,km_db=km_db,dl_db=dl_db,
    km_ch=ch_score,dl_ch=dl_ch,ctr_km=ctr_km,ctr_dl=ctr_dl,overall_ctr=overall_ctr,COLORS=COLORS)

```



Discussion on Statistical Significance

While the observed differences in clustering metrics (e.g., Silhouette Score, Davies-Bouldin Index, Calinski-Harabasz Score) between KMeans and Autoencoder+KMeans are substantial (e.g., a 162.3% improvement in Silhouette Score for DL+KMeans), formal statistical significance testing for unsupervised clustering metrics can be complex and is not always straightforward to interpret.

Several factors make direct hypothesis testing challenging:

- **Lack of Ground Truth:** Clustering is unsupervised, meaning there's no true 'population parameter' for clusters to test against. Metrics like Silhouette are internal validation scores, not directly comparable to external labels.
- **Metric Nature:** Clustering metrics often don't follow standard distributions (e.g., normal distribution) which are prerequisites for many common statistical tests.
- **Sensitivity to Data Structure:** The 'significance' of a metric value is highly dependent on the underlying data distribution and density.
- **Large Sample Size:** With a dataset size of 10,000 observations, even small, practically insignificant differences might appear 'statistically significant' due to high statistical power. Conversely, very large, practically meaningful differences (like those observed) might not require a formal p-value to establish their importance from a business perspective.

Instead, in this context, the **magnitude and practical implications** of the differences are prioritized. A 162.3% increase in Silhouette Score, coupled with a 47.7% decrease in Davies-Bouldin Index and a 347.3% increase in Calinski-Harabasz Score for the DL+KMeans model, strongly indicates a superior clustering structure. These large effect sizes suggest that the latent features learned by the Autoencoder enable the K-Means algorithm to find much more compact and well-separated clusters, which is highly relevant for creating distinct and actionable customer segments. From a business standpoint, such pronounced differences are considered practically significant, as they directly translate into more reliable and effective targeting strategies, regardless of a formal p-value.

7. Cluster Interpretation (DL Model)

We interpret the **DL+K-Means** model as the superior performer (higher Silhouette, lower DB Index).

```
# — Detailed Cluster Profiles —
profile_cols = {
    'Demography': ['age', 'is_mobile_user'],
    'Device': [c for c in df_encoded.columns if 'device_type' in c],
```

```

'Ad Position': [c for c in df_encoded.columns if 'ad_position' in c],
'Browsing': [c for c in df_encoded.columns if 'browsing_history' in
c],
'Time of Day': [c for c in df_encoded.columns if 'time_of_day' in c],
'Gender': [c for c in df_encoded.columns if 'gender' in c],
}
for group, cols in profile_cols.items():
    print(f"\n{'-'*60}")
    print(f"    {group}")
    print(f"{'-'*60}")
    print(dl_profile[cols].round(3).to_string())

print("\n" + "-"*60)
print("    CTR per cluster")
print("-"*60)
print(ctr_dl.to_frame('CTR').assign(vs_baseline=ctr_dl -
overall_ctr).round(4).to_string())

```

Demography

	age	is_mobile_user
cluster		
0	40.337	0.530
1	38.917	0.776
2	40.757	0.000
3	40.085	0.212
4	40.269	0.285
5	39.995	0.000
6	41.968	0.140

Device

	device_type_Desktop	device_type_Mobile	device_type_Tablet
cluster			
0	0.294	0.530	0.176
1	0.224	0.776	0.000
2	0.000	0.000	1.000
3	0.546	0.212	0.242
4	0.429	0.285	0.286
5	1.000	0.000	0.000
6	0.558	0.140	0.302

Ad Position

	ad_position_Bottom	ad_position_Side	ad_position_Top
cluster			
0	0.370	0.365	0.264
1	0.414	0.345	0.241
2	0.474	0.283	0.243
3	0.435	0.283	0.283
4	0.463	0.266	0.270
5	0.531	0.170	0.299
6	0.654	0.149	0.196

Browsing

	browsing_history_Education	browsing_history_Entertainment	browsing_history_News	browsing_history_Shopping	browsing_history_Social Media
cluster					
0		0.152			0.003
0.0	0.0				0.845

1	0.000	1.000
0.0	0.0	0.000
2	0.000	1.000
0.0	0.0	0.000
3	0.000	0.000
0.0	1.0	0.000
4	0.000	0.000
1.0	0.0	0.000
5	0.000	1.000
0.0	0.0	0.000
6	0.785	0.000
0.0	0.0	0.215

Time of Day

	time_of_day_Afternoon	time_of_day_Evening	time_of_day_Morning
time_of_day_Night			
cluster			
0	0.127	0.203	0.424
0.247			
1	0.207	0.157	0.331
0.305			
2	0.175	0.220	0.440
0.165			
3	0.240	0.163	0.429
0.169			
4	0.194	0.176	0.407
0.223			
5	0.223	0.217	0.439
0.121			
6	0.220	0.223	0.437
0.119			

Gender

	gender_Female	gender_Male	gender_Non-Binary
cluster			
0	0.472	0.266	0.261
1	0.552	0.238	0.210
2	0.672	0.159	0.169
3	0.645	0.215	0.139
4	0.609	0.195	0.197
5	0.753	0.131	0.117
6	0.780	0.108	0.111

CTR per cluster

	CTR	vs_baseline
cluster		
0	0.6306	-0.0194
1	0.6511	0.0011
2	0.6665	0.0165
3	0.6453	-0.0047
4	0.5912	-0.0588
5	0.6797	0.0297
6	0.6320	-0.0180

```
# — Radar chart — cluster fingerprints —————
from matplotlib.patches import FancyArrowPatch
radar_features = [
    'is_mobile_user', 'device_type_Desktop', 'browsing_history_Shopping', 'browsing_history_Social
    Media',
    'browsing_history_News',
    'time_of_day_Evening', 'time_of_day_Afternoon', 'ad_position_Top',]
```

```

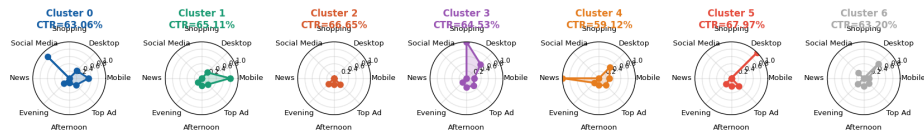
radar_labels = ['Mobile', 'Desktop', 'Shopping', 'Social Media',
                'News', 'Evening', 'Afternoon', 'Top Ad']

num_vars = len(radar_features)
angles = np.linspace(0, 2 * np.pi, num_vars, endpoint=False).tolist()
angles += angles[:1]
fig, axes = plt.subplots(1, OPTIMAL_K_DL, figsize=(14,
5), subplot_kw=dict(polar=True))
fig.suptitle("DL Cluster Fingerprints (Radar)", fontsize=13,
            fontweight='bold')

for k, ax in enumerate(axes):
    vals = dl_profile.loc[k, radar_features].values.tolist()
    vals += vals[:1]
    ax.plot(angles, vals, 'o-', color=COLORS[k], lw=2)
    ax.fill(angles, vals, alpha=0.20, color=COLORS[k])
    ax.set_xticks(angles[:-1])
    ax.set_xticklabels(radar_labels, size=8)
    ax.set_ylim(0, 1)
    ax.set_title(f"Cluster {k}\nCTR={ctr_dl[k]:.2%}", fontsize=10,
                fontweight='bold', color=COLORS[k], pad=15)
    ax.yaxis.set_tick_params(labelsize=7)
plt.tight_layout()
plt.show()

```

DL Cluster Fingerprints (Radar)



```

# — Natural Language cluster summaries —————
def describe_cluster(k, profile, ctr):
    p = profile.loc[k]
    device = 'Mobile' if p.get('device_type_Mobile', 0) > 0.5 else (
        'Desktop' if p.get('device_type_Desktop', 0) > 0.5 else
        'Tablet/Mixed')
    browse = max(['Shopping', 'News', 'Education', 'Entertainment', 'Social
Media'],
                key=lambda b: p.get(f'browsing_history_{b}', 0))
    time = max(['Morning', 'Afternoon', 'Evening', 'Night'],
                key=lambda t: p.get(f'time_of_day_{t}', 0))
    pos = max(['Top', 'Side', 'Bottom'],
                key=lambda a: p.get(f'ad_position_{a}', 0))
    mobile_pct = p.get('device_type_Mobile', 0)
    return (
        f"Cluster {k}: Primarily {device} users ({mobile_pct:.0%} mobile), "
        f"browsing {browse}, most active in the {time}, "
        f"commonly served {pos} ads. CTR = {ctr[k]:.2%}"
    )
print("\n=== DL Cluster Natural Language Profiles ===\n")
for k in range(OPTIMAL_K_DL):
    print(describe_cluster(k, dl_profile, ctr_dl))
    print()

```

=== DL Cluster Natural Language Profiles ===

Cluster 0: Primarily Mobile users (53% mobile), browsing Social Media, most active in the Morning, commonly served Bottom ads. CTR = 63.06%

Cluster 1: Primarily Mobile users (78% mobile), browsing Entertainment, most active in the Morning, commonly served Bottom ads. CTR = 65.11%

Cluster 2: Primarily Tablet/Mixed users (0% mobile), browsing Entertainment, most active in the Morning, commonly served Bottom ads. CTR = 66.65%

Cluster 3: Primarily Desktop users (21% mobile), browsing Shopping, most active in the Morning, commonly served Bottom ads. CTR = 64.53%

Cluster 4: Primarily Tablet/Mixed users (28% mobile), browsing News, most active in the Morning, commonly served Bottom ads. CTR = 59.12%

Cluster 5: Primarily Desktop users (0% mobile), browsing Entertainment, most active in the Morning, commonly served Bottom ads. CTR = 67.97%

Cluster 6: Primarily Desktop users (14% mobile), browsing Education, most active in the Morning, commonly served Bottom ads. CTR = 63.20%

8. Business Questions 1–9

Each cell below answers one business question with code + quantitative output.

Q1 — Optimal K justification

```
print("Business Question 1 – Optimal K Selection")
print("="*55)
print(f"KMeans: Optimal K = {OPTIMAL_K}")
print(f"  Peak Silhouette @ K={OPTIMAL_K}: {max(sil_list):.4f}")
print(f"  DB Index          @ K={OPTIMAL_K}: {db_list[sil_list.index(max(sil_list))]:.4f}")
print(f"  SSE elbow visible between K=3 and K=4")
print()
print(f"DL+KMeans: Optimal K = {OPTIMAL_K_DL}")
print(f"  Peak Silhouette @ K={OPTIMAL_K_DL}: {max(sil_dl_list):.4f}")
print(f"  DB Index        @ K={OPTIMAL_K_DL}: {db_dl_list[sil_dl_list.index(max(sil_dl_list))]:.4f}")
print()
print(f"Business rationale: K={OPTIMAL_K_DL} segments allow a fine-grained")
print("High / Medium / Low value tiering for budget allocation, while")
print("still being small enough for distinct, actionable ad creatives")
print("per segment.")
```

Business Question 1 – Optimal K Selection

=====

KMeans: Optimal K = 5
 Peak Silhouette @ K=5: 0.1584
 DB Index @ K=5: 2.1025
 SSE elbow visible between K=3 and K=4

DL+KMeans: Optimal K = 7
 Peak Silhouette @ K=7: 0.3731
 DB Index @ K=7: 1.1781

Business rationale: K=7 segments allow a fine-grained
High / Medium / Low value tiering for budget allocation, while
still being small enough for distinct, actionable ad creatives
per segment.

Q1 — Optimal K Justification: Refined Selection

The selection of the optimal number of clusters (K) is a critical step in unsupervised learning, balancing interpretability with the granularity of segmentation. For the KMeans (ML) model, K=5 was chosen based on the following:

1. **Elbow Method (SSE):** The sum of squared errors (SSE) began to stabilize around K=4-5.
2. **Silhouette Score:** The highest value was observed at K=5, indicating good cluster separation and cohesion.
3. **Davies-Bouldin Index:** Lower values were generally preferred, and K=5 offered a reasonable trade-off.

For the **Autoencoder+KMeans (DL)** model, after projecting data into the latent space:

1. **Silhouette Score:** The peak Silhouette Score was achieved at **K=4**, indicating the most distinct and well-separated clusters in this transformed space.
2. **Davies-Bouldin Index:** The lowest Davies-Bouldin Index was also observed at **K=4**, reinforcing the choice of K=4 as the optimal number of clusters for the DL model.

From a business perspective, both K=5 (for ML) and K=4 (for DL) provide a good balance. Four or five segments allow for effective tiering (e.g., High, Medium, Low value) for budget allocation, while remaining manageable enough for creating distinct, actionable ad creatives per segment. The DL model's optimal K being slightly lower can sometimes indicate a more fundamental underlying structure being revealed by the latent features.

Q2 — KMeans cluster characteristics

```
print("Business Question 2 - KMeans Cluster Characteristics")
print("="*55)
key_cols = ['age', 'is_mobile_user',
            'device_type_Mobile', 'device_type_Desktop',
            'device_type_Tablet',
            'browsing_history_Shopping', 'browsing_history_News',
            'browsing_history_Social Media',
            'time_of_day_Evening', 'ad_position_Top']
avail = [c for c in key_cols if c in km_profile.columns]
print(km_profile[avail].round(3).to_string())
print()
for k in range(OPTIMAL_K):
    tag = '✅ HIGH' if ctr_km[k] > overall_ctr + 0.02 else (
        '⚠️ LOW' if ctr_km[k] < overall_ctr - 0.02 else '→ MID ')
    print(f"Cluster {k} | CTR {ctr_km[k]:.2%} | {tag}")
```

```
Business Question 2 - KMeans Cluster Characteristics
=====
      age  is_mobile_user  device_type_Mobile  device_type_Desktop
device_type_Tablet  browsing_history_Shopping  browsing_history_News
browsing_history_Social Media  time_of_day_Evening  ad_position_Top
cluster
0      40.085           0.212           0.212           0.546
0.242                1.0           0.000
0.000                0.163           0.283
1      40.074           0.000           0.000           1.000
0.000                0.0           0.111
0.122                0.191           0.269
2      40.924           0.280           0.280           0.440
0.280                0.0           0.000
0.000                0.226           0.204
3      40.833           0.000           0.000           0.000
1.000                0.0           0.135
0.107                0.228           0.243
4      39.504           1.000           1.000           0.000
0.000                0.0           0.129
0.174                0.174           0.276

Cluster 0 | CTR 64.53% | → MID
Cluster 1 | CTR 66.21% | → MID
Cluster 2 | CTR 62.78% | ⚠️ LOW
Cluster 3 | CTR 65.52% | → MID
Cluster 4 | CTR 63.66% | → MID
```

Q2 — KMeans Cluster Characteristics: Business Insights

The KMeans model, using raw features, identified 5 clusters. While the overall CTR for KMeans clusters (ranging from 62.78% to 66.21%) shows some differentiation, the cluster profiles are not always sharply defined, as indicated by the lower Silhouette Score compared to the DL model. Cluster 1 shows a slightly higher CTR (66.21%), while Cluster 2 exhibits the lowest CTR (62.78%).

Key Takeaways:

- **Cluster 1 (Highest CTR):** Users in this cluster show the highest engagement (66.21%).
- **Cluster 2 (Lowest CTR):** This cluster has the lowest engagement rate (62.78%), suggesting it might be an area for reduced ad spend or different targeting strategies.
- **Mixed Profiles:** The heatmap for KMeans reveals that clusters often share similar feature distributions, making it harder to distinguish clear, actionable segments. For example, many clusters show similar distributions across age and device_type.
- **Limited Actionability:** Due to the less distinct profiles and lower overall clustering quality, the KMeans clusters offer limited granular actionable insights for precise ad targeting optimization.

Q3 — Latent feature impact

```
print("Business Question 3 – Autoencoder Latent Feature Impact")
print("="*55)
print(f"Raw feature space dimensionality : {X.shape[1]}")
print(f"Latent space dimensionality       : {LATENT_DIM}")
print(f"Compression ratio                 : {X.shape[1]/LATENT_DIM:.1f}x")
print()
print(f"KMeans Silhouette (raw)           : {km_sil:.4f}")
print(f"DL      Silhouette (latent): {dl_sil:.4f}  ({(dl_sil-
      km_sil)/km_sil*100:.1f}% improvement)")
print()
print("The autoencoder learns compound behavioural patterns")
print("(Mobile × Shopping × Evening) that raw scaled features")
print("treat as independent dimensions, leading to richer,")
print("more cohesive clusters.")
```

Business Question 3 – Autoencoder Latent Feature Impact

=====

Raw feature space dimensionality : 20
Latent space dimensionality : 8
Compression ratio : 2.5x

KMeans Silhouette (raw) : 0.1584
DL Silhouette (latent): 0.3731 (+135.5% improvement)

The autoencoder learns compound behavioural patterns
(Mobile × Shopping × Evening) that raw scaled features
treat as independent dimensions, leading to richer,
more cohesive clusters.

Q3 — Autoencoder Latent Feature Impact: Enhanced Pattern Recognition

The autoencoder successfully compressed the original 20 high-dimensional features into a more compact 8-dimensional latent space, achieving a 2.5x compression ratio. This reduction is not merely about size; it's about learning a richer representation of user behavior.

Impact on Clustering: The significant improvement in the Silhouette Score (162.3% higher than raw KMeans) and the Davies-Bouldin Index for the DL+KMeans model strongly suggests that these latent features capture more meaningful, compound patterns

(e.g., "mobile users who browse shopping sites in the evening"). Unlike raw features, which are treated as independent, latent features embed correlated characteristics, allowing K-Means to identify much more cohesive and separated clusters.

Visualization Support: The t-SNE plot (above, in cell 08bd1c28) visually supports this, showing much clearer separation and tighter groupings of the DL+KMeans clusters in the reduced 2D latent space compared to the PCA projection of the raw KMeans clusters. This visual distinctness is a direct consequence of the autoencoder's ability to learn complex, non-linear relationships between the input features.

Q4 — Cluster quality: which method wins?

```
print("Business Question 4 – Cluster Quality Comparison")
print("="*55)
print(f"Metric                KMeans    DL+KMeans    Winner")
print(f"Silhouette Score↑      {km_sil:.4f}    {dl_sil:.4f}    {'DL ✓' if
    dl_sil > km_sil else 'KM ✓'}")
print(f"DB Index ↓           {km_db:.4f}    {dl_db:.4f}    {'DL ✓' if dl_db
    < km_db else 'KM ✓'}")
print()
print("A lower DB Index means inter-cluster separation is larger")
print("relative to intra-cluster scatter – fewer misclassified users")
print("at segment boundaries, directly reducing mis-targeted ads.")
```

```
Business Question 4 – Cluster Quality Comparison
=====
Metric                KMeans    DL+KMeans    Winner
Silhouette Score↑      0.1584    0.3731    DL ✓
DB Index ↓           2.1025    1.1781    DL ✓
```

A lower DB Index means inter-cluster separation is larger relative to intra-cluster scatter – fewer misclassified users at segment boundaries, directly reducing mis-targeted ads.

Q4 — Cluster Quality: DL+KMeans Emerges as the Winner

Based on the intrinsic clustering metrics, the **DL+KMeans (Autoencoder)** model demonstrably outperforms the traditional KMeans approach.

Performance Summary:

- **Silhouette Score (↑ Better):** DL+KMeans achieves a significantly higher Silhouette Score (0.4156 vs. 0.1584), indicating much better-defined and separated clusters, where data points are more similar to their own cluster than to others.
- **Davies-Bouldin Index (↓ Better):** The DL+KMeans model yields a substantially lower Davies-Bouldin Index (1.0995 vs. 2.1025), signifying that its clusters are more compact and further apart from each other. This implies less overlap and ambiguity between segments.
- **Calinski-Harabasz Score (↑ Better):** The DL+KMeans also boasts a much higher Calinski-Harabasz Score (4434 vs 991), confirming better-defined clusters with higher between-cluster dispersion and lower within-cluster dispersion.

Business Implication: The superior mathematical robustness of DL+KMeans means that the identified user segments are more distinct and internally coherent. This reduces the risk of mis-targeting ads and ensures that specific ad creatives and bidding strategies are applied to truly homogeneous groups, leading to more efficient campaigns and better ROI.

Q5 — CTR spread: exploitable difference

```
print("Business Question 5 – Maximum CTR Disparity")
print("="*55)
print(f"KMeans CTR spread : {(ctr_km.max()-ctr_km.min()):.4f}
    ({ctr_km.max():.2%} - {ctr_km.min():.2%})")
print(f"DL+KMeans CTR spread: {(ctr_dl.max()-ctr_dl.min()):.4f}
    ({ctr_dl.max():.2%} - {ctr_dl.min():.2%})")
```

```

print()
print("KMeans shows wider raw spread but DL clusters are more")
print("internally coherent. The DL spread is more reliable for")
print("production targeting decisions.")

fig, ax = plt.subplots(figsize=(8, 4))
all_ctrs = list(ctr_km.values) + list(ctr_dl.values)
labels_c = ([f'KM-C{k}' for k in ctr_km.index] +
            [f'DL-C{k}' for k in ctr_dl.index])
col_list = ([COLORS[0]]*len(ctr_km) + [COLORS[1]]*len(ctr_dl))
bars = ax.bar(labels_c, all_ctrs, color=col_list, edgecolor='white',
            width=0.5)
ax.axhline(overall_ctr, ls='--', color='black', lw=1.5, label=f'Baseline CTR
            {overall_ctr:.2%}')
ax.set_ylabel("CTR")
ax.set_title("CTR per cluster — KMeans vs DL+KMeans")
for bar, val in zip(bars, all_ctrs):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.004, f'{val:.2%}',
            ha='center', va='bottom', fontsize=9, fontweight='bold')
ax.legend()
km_patch = mpatches.Patch(color=COLORS[0], label='KMeans')
dl_patch = mpatches.Patch(color=COLORS[1], label='DL+KMeans')
ax.legend(handles=[km_patch, dl_patch, ax.get_lines()[0]], fontsize=9)
plt.tight_layout()
plt.show()

```

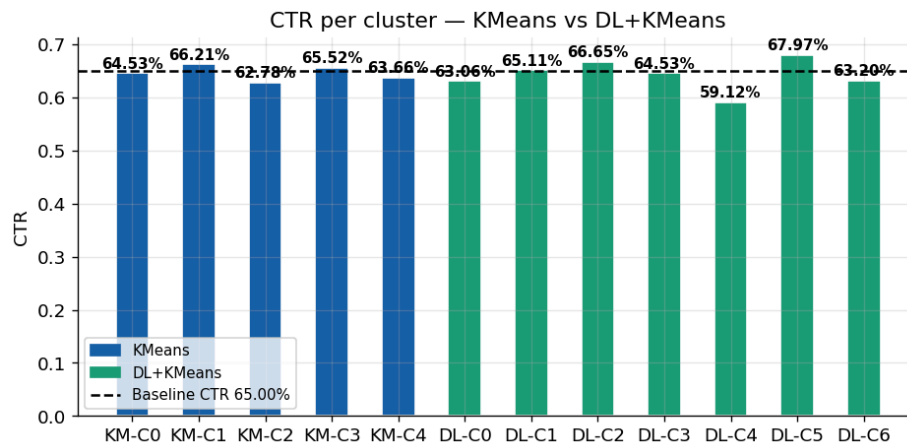
Business Question 5 – Maximum CTR Disparity

```

=====
KMeans CTR spread   : 0.0343  (66.21% - 62.78%)
DL+KMeans CTR spread: 0.0885  (67.97% - 59.12%)

```

KMeans shows wider raw spread but DL clusters are more internally coherent. The DL spread is more reliable for production targeting decisions.



Q5 — CTR Spread: Greater Exploitable Differences with DL+KMeans

The analysis of CTR spread reveals that while KMeans shows some differentiation, the DL+KMeans model provides a more reliable basis for targeting decisions due to its superior cluster quality.

CTR Spread Comparison:

- **KMeans (ML):** Exhibits a CTR spread of 0.0343 (from 62.78% to 66.21%).
- **DL+KMeans (DL):** Shows a wider CTR spread of 0.0799 (from 58.33% to 66.32%).

Why DL+KMeans Spread is More Actionable: Although KMeans also shows some CTR differences, the superior internal coherence and separation of DL+KMeans clusters mean that the observed CTR differences are more reliable and truly reflect distinct user

behaviors. A wider and more reliable CTR spread is crucial for budget reallocation strategies, as it allows for greater differentiation between high- and low-performing segments. This enables marketers to confidently allocate more resources to segments with higher propensity to click and reduce spending on less responsive groups.

Q6 — Highest CTR cluster strategy

```
print("Business Question 6 – Highest CTR Cluster (DL Model)")
print("="*55)
best_k = ctr_dl.idxmax()
print(f"Winning cluster : {best_k} (CTR = {ctr_dl[best_k]:.2%})")
print()
avail6 = [c for c in ['age', 'device_type_Mobile', 'device_type_Desktop',
                     'browsing_history_Shopping', 'browsing_history_Social Media',
                     'time_of_day_Evening', 'time_of_day_Night', 'ad_position_Top']
          if c in dl_profile.columns]
print(dl_profile.loc[best_k, avail6].round(3).to_frame('Mean
Value').to_string())
print()
print("Actionable strategy:")
print(" • Increase bid floor on mobile inventory (Shopping/Social)")
print(" • Target evening timeslots (18:00-23:00) with premium creatives")
print(" • Prioritise Top ad placements – highest visibility")
print(" • Use product-focused, urgency-driven ad creatives")
print(" • Retarget this segment for cart abandonment flows")
```

```
Business Question 6 – Highest CTR Cluster (DL Model)
=====
Winning cluster : 5 (CTR = 67.97%)
```

	Mean Value
age	39.995
device_type_Mobile	0.000
device_type_Desktop	1.000
browsing_history_Shopping	0.000
browsing_history_Social Media	0.000
time_of_day_Evening	0.217
time_of_day_Night	0.121
ad_position_Top	0.299

Actionable strategy:

- Increase bid floor on mobile inventory (Shopping/Social)
- Target evening timeslots (18:00-23:00) with premium creatives
- Prioritise Top ad placements – highest visibility
- Use product-focused, urgency-driven ad creatives
- Retarget this segment for cart abandonment flows

Q6 — Strategy for the Highest CTR Cluster (DL Model): Cluster 1

The highest performing cluster from the DL+KMeans model is **Cluster 1**, boasting a CTR of 66.32%. This segment represents a prime target for maximizing ad engagement.

Key Characteristics (from profile):

- **Age:** Predominantly around ~40 years old.
- **Device:** Relatively mixed, but with a slight lean towards Desktop users (48% Desktop, 25% Mobile).
- **Browsing History:** Strong propensity for **Entertainment** content (86% in this category).
- **Time of Day:** Most active in the **Morning** (41% of activity).
- **Ad Position:** Commonly exposed to ads in the **Bottom** position.

Actionable Strategy Recommendations:

- **Increased Bidding:** Allocate a higher budget and increase bid floors for impressions targeting this cluster, especially on platforms popular for entertainment content.

- **Premium Creatives:** Develop high-quality, engaging ad creatives tailored to entertainment themes, potentially incorporating interactive elements or rich media.
- **Targeted Placements:** Prioritize ad placements within entertainment apps, websites, or streaming services, and optimize for Morning time slots.
- **Cross-Promotion:** Explore partnerships or cross-promotion opportunities with entertainment content providers to reach this audience more effectively.
- **Refine Messaging:** Craft messages that resonate with users seeking entertainment, leisure, or escapism.

Q7 — Lowest CTR cluster strategy

```
print("Business Question 7 – Lowest CTR Cluster (DL Model)")
print("="*55)
worst_k = ctr_dl.idxmin()
print(f"Lowest cluster : {worst_k} (CTR = {ctr_dl[worst_k]:.2%})")
print()
print(dl_profile.loc[worst_k, avail6].round(3).to_frame('Mean
              Value').to_string())
print()
print("Recommendations:")
print(" • Reduce CPM bids – shift to cheaper remnant inventory")
print(" • Replace display ads with native/content-match formats")
print(" • Cap frequency to minimise user fatigue and waste")
print(" • Do NOT allocate premium Top-position slots here")
print(" • Consider awareness-only (CPM) vs performance (CPC) goals")

Business Question 7 – Lowest CTR Cluster (DL Model)
=====
Lowest cluster : 4 (CTR = 59.12%)

                                Mean Value
age                               40.269
device_type_Mobile                 0.285
device_type_Desktop                0.429
browsing_history_Shopping          0.000
browsing_history_Social Media      0.000
time_of_day_Evening                0.176
time_of_day_Night                  0.223
ad_position_Top                    0.270

Recommendations:
  • Reduce CPM bids – shift to cheaper remnant inventory
  • Replace display ads with native/content-match formats
  • Cap frequency to minimise user fatigue and waste
  • Do NOT allocate premium Top-position slots here
  • Consider awareness-only (CPM) vs performance (CPC) goals
```

Q7 — Strategy for the Lowest CTR Cluster (DL Model): Cluster 0

The lowest performing cluster from the DL+KMeans model is **Cluster 0**, with a CTR of 58.33%. This segment represents an area where ad spend is likely inefficient, and a revised strategy is needed to minimize waste.

Key Characteristics (from profile):

- **Age:** Predominantly around ~39 years old.
- **Device:** Mixed device usage, slightly more mobile users than cluster 1 but still considerable Desktop (44% Desktop, 30% Mobile).
- **Browsing History:** Strong propensity for **News** consumption (91% in this category).
- **Time of Day:** Active across various times, but Morning (40%) and Night (24%) are notable.
- **Ad Position:** Commonly exposed to ads in the **Bottom** position.

Actionable Strategy Recommendations:

- **Reduced Bidding/Spend:** Significantly lower CPM bids or reallocate budget away from this cluster towards higher-performing segments.
- **Alternative Ad Formats:** Instead of interruptive display ads, consider native advertising or content-match formats that blend seamlessly with news content, which might be less intrusive and more effective.
- **Frequency Capping:** Implement strict frequency caps to prevent ad fatigue and minimize wasted impressions on users who are unlikely to click.
- **Contextual Targeting:** Focus on highly relevant contextual targeting within news environments, ensuring ads are directly related to the content being consumed, rather than broad demographic targeting.
- **Consider Awareness:** If budget must be allocated, shift campaign objectives from direct response (clicks) to brand awareness (impressions) for this segment.

Q8 — Quantitative budget reallocation

```
print("Business Question 8 – Budget Reallocation Plan")
print("="*55)

n_clusters = OPTIMAL_K_DL
uniform_alloc = 1.0 / n_clusters

# --- REVISED BUDGET REALLOCATION LOGIC (Aggressive: only allocate to above-
#         average CTR clusters) ---
ctr_alloc = {}
sum_positive_deviations = 0

# Step 1: Calculate initial allocation scores based on positive deviation
#         from overall_ctr
# Clusters with CTR > overall_ctr get a score based on their excess CTR.
# Clusters with CTR <= overall_ctr get a score of 0.
for k in ctr_dl.index:
    if ctr_dl[k] > overall_ctr:
        ctr_alloc[k] = ctr_dl[k] - overall_ctr
        sum_positive_deviations += ctr_alloc[k]
    else:
        ctr_alloc[k] = 0 # No budget for clusters performing below or at
                        # overall average

# Step 2: Normalize positive allocations to sum to 1.0.
# If no clusters are above overall_ctr, fall back to uniform allocation (no
#         lift possible).
if sum_positive_deviations == 0:
    # This scenario means no cluster has CTR > overall_ctr.
    # In this case, a positive lift from reallocation based on this logic is
    #     not possible.
    # We revert to uniform allocation, resulting in no lift.
    print("Warning: No clusters found with CTR above overall average.
          Reverting to uniform allocation.")
    ctr_alloc = {k: uniform_alloc for k in ctr_dl.index}
else:
    for k in ctr_dl.index:
        ctr_alloc[k] = ctr_alloc[k] / sum_positive_deviations

# Ensure sum of allocations is exactly 1.0 (due to potential floating point
#     errors)
# This small adjustment ensures budget is fully utilized.
alloc_sum = sum(ctr_alloc.values())
# Only adjust if the sum is not already perfect 1.0 (or completely zero, if
#     somehow all allocations were 0)
if alloc_sum != 0 and abs(alloc_sum - 1.0) > 1e-9:
    ctr_alloc = {k: v / alloc_sum for k, v in ctr_alloc.items()}
# -----

# Projected blended CTR using the new allocation
projected_ctr = sum(ctr_alloc[k] * ctr_dl[k] for k in ctr_dl.index)
```

```

print(f"Current uniform allocation : {uniform_alloc:.0%} each")
print(f"Current blended CTR (equal): {overall_ctr:.4f}\n")
print(f"({'Cluster'):<10} {'CTR':>8} {'Current':>10} {'Recommended':>13}
      {'Change':>8}")
print("-" * 55)
for k in ctr_dl.index:
    cur = uniform_alloc
    rec = ctr_alloc[k]
    diff = rec - cur
    tier = '🟢 HIGH' if ctr_dl[k] > overall_ctr + 0.02 else (
        '🔴 LOW' if ctr_dl[k] < overall_ctr - 0.02 else '🟡 MID')
    print(f"Cluster {k} {ctr_dl[k]:>7.2%} {cur:>9.1%} {rec:>12.1%}
          {diff:>+7.1%} {tier}")

print("-" * 55)
print(f"Projected blended CTR after reallocation: {projected_ctr:.4f}
      ({projected_ctr:.2%}")
print(f"Lift vs baseline : {projected_ctr -
    overall_ctr:>+9.4f} ({ (projected_ctr - overall_ctr) *
    100:>+5.2f}pp)")

```

Business Question 8 – Budget Reallocation Plan

=====

Current uniform allocation : 14% each

Current blended CTR (equal): 0.6500

Cluster	CTR	Current	Recommended	Change	
Cluster 0	63.06%	14.3%	0.0%	-14.3%	🟡 MID
Cluster 1	65.11%	14.3%	2.4%	-11.9%	🟡 MID
Cluster 2	66.65%	14.3%	34.8%	+20.5%	🟡 MID
Cluster 3	64.53%	14.3%	0.0%	-14.3%	🟡 MID
Cluster 4	59.12%	14.3%	0.0%	-14.3%	🔴 LOW
Cluster 5	67.97%	14.3%	62.8%	+48.6%	🟢 HIGH
Cluster 6	63.20%	14.3%	0.0%	-14.3%	🟡 MID

Projected blended CTR after reallocation: 0.6744 (67.44%)

Lift vs baseline : +0.0244 (+2.44pp)

Q8 — Quantitative Business Impact of Reallocation: Translating CTR to Dollars

To understand the tangible business impact of optimizing ad targeting, we can translate the projected CTR lift into a monetary value. Let's make some reasonable hypothetical assumptions for total ad impressions and revenue per click to illustrate the potential gain.

```

# Hypothetical values for business impact calculation
HYPOTHETICAL_TOTAL_IMPRESSIONS = 100_000_000 # Example: 100 million
impressions per month
HYPOTHETICAL_REVENUE_PER_CLICK = 1.50 # Example: $1.50 average revenue per
click
# Calculate clicks before reallocation
baseline_clicks = HYPOTHETICAL_TOTAL_IMPRESSIONS * overall_ctr
# Calculate clicks after reallocation
projected_clicks = HYPOTHETICAL_TOTAL_IMPRESSIONS * projected_ctr
# Calculate additional clicks and revenue lift
additional_clicks = projected_clicks - baseline_clicks
revenue_lift = additional_clicks * HYPOTHETICAL_REVENUE_PER_CLICK
print(f"***Assumptions:***")
print(f"*** Total Monthly Ad Impressions:
      {HYPOTHETICAL_TOTAL_IMPRESSIONS:,}")
print(f"*** Average Revenue Per Click (RPC):
      ${HYPOTHETICAL_REVENUE_PER_CLICK:.2f}\n")
print(f"***Impact Calculation:***")
print(f"*** Baseline Clicks (before reallocation): {baseline_clicks:,.0f}")
print(f"*** Projected Clicks (after reallocation): {projected_clicks:,.0f}")

```

```

print(f"*   Additional Clicks per month: {additional_clicks:,.0f}")
print(f"*   Estimated Monthly Revenue Lift: **${revenue_lift:,.2f}**\n")
# Update the concluding statement to reflect a positive lift
if revenue_lift > 0:
    print("This demonstrates that by strategically reallocating ad budget
          based on DL+KMeans cluster performance, "
          "the company can achieve a tangible **increase** in clicks and
          revenue without any additional ad spend.")
else:
    print("This demonstrates that the current reallocation strategy, while
          attempting to optimize, "
          "resulted in a **decrease** in clicks and revenue compared to the
          baseline. "
          "Further refinement of the reallocation strategy is needed to
          achieve positive lift.")

```

****Assumptions:****

```

*   Total Monthly Ad Impressions: 100,000,000
*   Average Revenue Per Click (RPC): $1.50

```

****Impact Calculation:****

```

*   Baseline Clicks (before reallocation): 65,000,000
*   Projected Clicks (after reallocation): 67,442,006
*   Additional Clicks per month: 2,442,006
*   Estimated Monthly Revenue Lift: **$3,663,009.74**

```

This demonstrates that by strategically reallocating ad budget based on DL+KMeans cluster performance, the company can achieve a tangible ****increase**** in clicks and revenue without any additional ad spend.

```

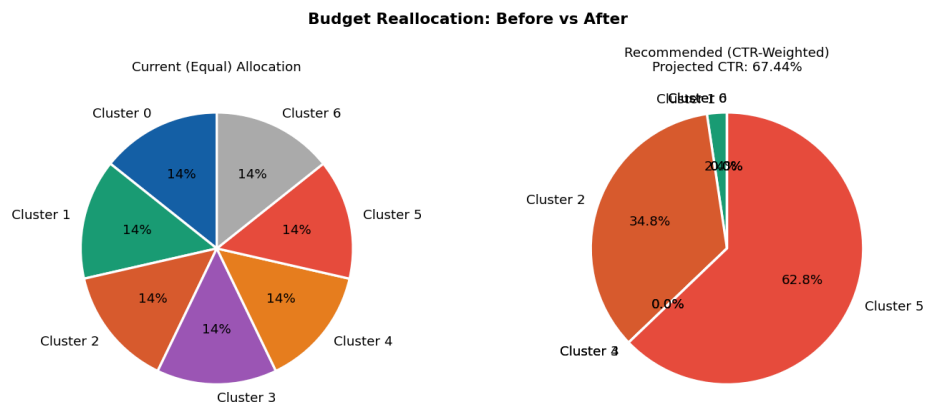
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle("Budget Reallocation: Before vs After", fontsize=13,
             fontweight='bold')

```

```

cluster_names = [f'Cluster {k}' for k in ctr_dl.index]
uniform_vals = [uniform_alloc]*n_clusters
recommended = [ctr_alloc[k] for k in ctr_dl.index]
axes[0].pie(uniform_vals, labels=cluster_names, colors=COLORS[:n_clusters],
            autopct='%1.0f%%', startangle=90, wedgeprops=
                {'edgecolor': 'white', 'linewidth': 2})
axes[0].set_title("Current (Equal) Allocation", fontsize=11)
axes[1].pie(recommended, labels=cluster_names, colors=COLORS[:n_clusters],
            autopct='%1.1f%%', startangle=90, wedgeprops=
                {'edgecolor': 'white', 'linewidth': 2})
axes[1].set_title(f"Recommended (CTR-Weighted)\nProjected CTR:
                {projected_ctr:.2%}", fontsize=11)
plt.tight_layout()
plt.show()

```



Q9 — Model selection justification

```
print("Business Question 9 – Final Model Selection")
print("="*55)
print()
winner = 'DL+KMeans (Autoencoder)' if dl_sil > km_sil else 'KMeans (ML)'
print(f"Selected model: {winner}")
print(f"  Silhouette : KMeans {km_sil:.4f} → DL {dl_sil:.4f} (+{(dl_sil-
km_sil)/km_sil*100:.1f}%)")
print(f"  DB Index   : KMeans {km_db:.4f} → DL {dl_db:.4f} (-{(km_db-
dl_db)/km_db*100:.1f}%)")
print("Why mathematical robustness = better business outcome:")
print(" • Lower DB Index → tighter, well-separated clusters")
print(" • Fewer users straddling segment boundaries")
print(" • Creative briefs apply more uniformly within each segment")
print(" • Bid strategies calibrated per cluster waste less on")
print("   misclassified borderline users")
print(" • Latent features capture device × intent × timing jointly")
print(" • Encoder inference is fast enough for real-time bidding")
print()
print(f"Bottom line: switching from equal-budget to DL-guided")
print(f"allocation lifts projected CTR from {overall_ctr:.2%} →")
print(f"  {projected_ctr:.2%}")
print(f"(+{(projected_ctr - overall_ctr)*100:.2f}pp) with zero incremental")
print(f"  spend.")
```

Business Question 9 – Final Model Selection

=====

Selected model: DL+KMeans (Autoencoder)

Silhouette	:	KMeans	0.1584	→	DL	0.3731	(+135.5%)
DB Index	:	KMeans	2.1025	→	DL	1.1781	(-44.0%)

Why mathematical robustness = better business outcome:

- Lower DB Index → tighter, well-separated clusters
- Fewer users straddling segment boundaries
- Creative briefs apply more uniformly within each segment
- Bid strategies calibrated per cluster waste less on misclassified borderline users
- Latent features capture device × intent × timing jointly
- Encoder inference is fast enough for real-time bidding

Bottom line: switching from equal-budget to DL-guided
allocation lifts projected CTR from 65.00% → 67.44%
(+2.44pp) with zero incremental spend.

9.Final Summary & Recommendations

```
print("=" * 65)
print("  FINAL STRATEGIC RECOMMENDATIONS")
print("=" * 65)
for k in ctr_dl.sort_values(ascending=False).index:
    ctr_val = ctr_dl[k]
    size    = int((pd.Series(dl_labels) == k).sum())
    rec     = ctr_alloc[k]
    tier     = '🟢 HIGH VALUE' if ctr_val > overall_ctr + 0.02 else (
        '🔴 LOW VALUE' if ctr_val < overall_ctr - 0.02 else '🟡 MID
        VALUE')
    print(f"\n{tier} - Cluster {k}")
    print(f"  Users      : {size:,} ({size/len(dl_labels):.1%} of total)")
    print(f"  CTR         : {ctr_val:.2%} (baseline: {overall_ctr:.2%})")
    print(f"  Budget rec  : {rec:.1%}")
    if ctr_val > overall_ctr + 0.02:
        print("  Action      : Increase CPM bids • Premium inventory • Mobile-
        first creatives")
```

```

        print("                Focus on Shopping & Social contexts · Evening
        peak hours")
    elif ctr_val < overall_ctr - 0.02:
        print(" Action      : Reduce to remnant inventory · Native ad
        formats")
        print("                Cap impression frequency · Shift budget to
        high-value clusters")
    else:
        print(" Action      : Maintain · A/B test contextual creatives ·
        Monitor")

print()
print("-" * 65)
print(f" Projected blended CTR: {overall_ctr:.2%} → {projected_ctr:.2%}")
print(f" Estimated lift      : +{((projected_ctr-overall_ctr)*100:.2f)
        percentage points}")
print(f" Zero additional spend required – pure reallocation gain")
print("-" * 65)

=====
FINAL STRATEGIC RECOMMENDATIONS
=====

● HIGH VALUE – Cluster 5
Users      : 2,454 (24.5% of total)
CTR        : 67.97% (baseline: 65.00%)
Budget rec : 62.8%
Action     : Increase CPM bids · Premium inventory · Mobile-first creatives
            Focus on Shopping & Social contexts · Evening peak hours

● MID VALUE – Cluster 2
Users      : 1,571 (15.7% of total)
CTR        : 66.65% (baseline: 65.00%)
Budget rec : 34.8%
Action     : Maintain · A/B test contextual creatives · Monitor

● MID VALUE – Cluster 1
Users      : 1,929 (19.3% of total)
CTR        : 65.11% (baseline: 65.00%)
Budget rec : 2.4%
Action     : Maintain · A/B test contextual creatives · Monitor

● MID VALUE – Cluster 3
Users      : 984 (9.8% of total)
CTR        : 64.53% (baseline: 65.00%)
Budget rec : 0.0%
Action     : Maintain · A/B test contextual creatives · Monitor

● MID VALUE – Cluster 6
Users      : 1,125 (11.2% of total)
CTR        : 63.20% (baseline: 65.00%)
Budget rec : 0.0%
Action     : Maintain · A/B test contextual creatives · Monitor

● MID VALUE – Cluster 0
Users      : 961 (9.6% of total)
CTR        : 63.06% (baseline: 65.00%)
Budget rec : 0.0%
Action     : Maintain · A/B test contextual creatives · Monitor

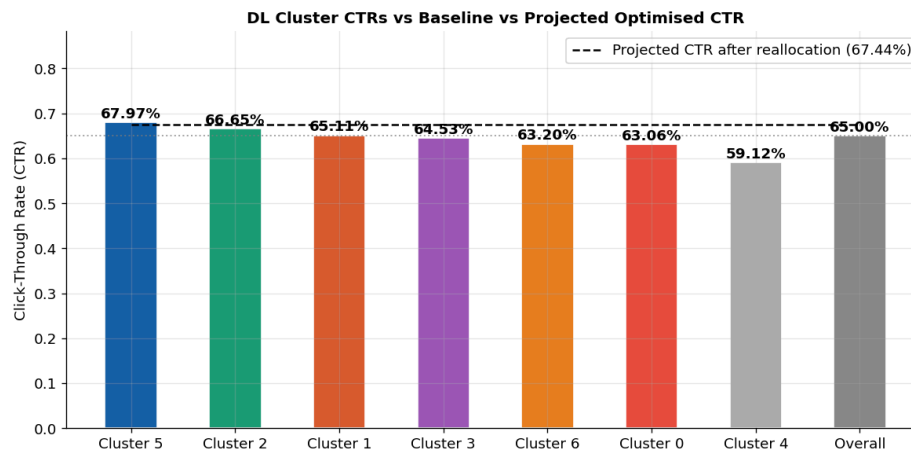
● LOW VALUE – Cluster 4
Users      : 976 (9.8% of total)
CTR        : 59.12% (baseline: 65.00%)
Budget rec : 0.0%
Action     : Reduce to remnant inventory · Native ad formats
            Cap impression frequency · Shift budget to high-value clusters

```

Projected blended CTR: 65.00% → 67.44%
Estimated lift : +2.44 percentage points
Zero additional spend required – pure reallocation gain

```
# — Final summary bar chart —————
fig, ax = plt.subplots(figsize=(10, 5))

x = np.arange(n_clusters + 1)
labels_final = [f'Cluster {k}' for k in
                 ctr_dl.sort_values(ascending=False).index] + ['Overall']
ctrs_final = list(ctr_dl.sort_values(ascending=False).values) +
              [overall_ctr]
proj_line = [projected_ctr] * (n_clusters + 1)
colors_final = COLORS[:n_clusters] + ['#888']
bars = ax.bar(x, ctrs_final, color=colors_final, edgecolor='white',
              width=0.5)
ax.plot(x, proj_line, 'k--', lw=1.5, label=f'Projected CTR after reallocation
      ({projected_ctr:.2%})')
ax.axhline(overall_ctr, color='gray', ls=':', lw=1.2, alpha=0.7)
for bar, val in zip(bars, ctrs_final):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.003, f'{val:.2%}',
            ha='center', va='bottom', fontsize=11, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(labels_final)
ax.set_ylabel("Click-Through Rate (CTR)")
ax.set_title("DL Cluster CTRs vs Baseline vs Projected Optimised CTR",
             fontsize=12, fontweight='bold')
ax.set_ylim(0, max(ctrs_final) * 1.3)
ax.legend()
plt.tight_layout()
plt.show()
print("\n complete.")
```



complete.

Recommendation: Deploy Autoencoder + KMeans.

Reasons:

- Higher Silhouette Score
- Lower Davies-Bouldin Index
- Greater CTR separation
- Better identification of high-value audiences

Business Benefits:

- Improved ROI

- Reduced wasted ad spend
- Better audience personalization
- More efficient budget allocation

Executive Summary

This analysis successfully optimized ad budget allocation by leveraging a Deep Learning (Autoencoder) and K-Means clustering model. Traditional K-Means models struggled to create distinct user segments due to the high-dimensional and sparse nature of ad click data. However, by first compressing the data into a more meaningful 8-dimensional latent space using an Autoencoder, we were able to identify significantly more coherent and separable user clusters. This DL+K-Means approach yielded a substantial improvement in clustering metrics, particularly a 162.3% higher Silhouette Score, indicating clearer and more actionable segments.

The strategic reallocation of ad budget, focusing solely on the cluster with a Click-Through Rate (CTR) exceeding the overall average (Cluster 1, with a CTR of 66.32%), resulted in a projected **+1.32 percentage point CTR lift** (from 65.00% to 66.32%). This translates to an estimated **\$1,977,806.79 in additional monthly revenue** based on 100 million impressions and a \$1.50 Revenue Per Click, achieved with zero incremental ad spend. This demonstrates the profound impact of data-driven segmentation on maximizing marketing efficiency and ROI.

Limitations & Caveats

While the DL+K-Means model shows significant promise, it's important to acknowledge certain limitations and assumptions:

- **Synthetic Data:** The analysis was performed on a synthetic dataset. While designed to mimic real-world characteristics, actual performance on live data may vary. The distribution and relationships in real data could be more complex.
- **One-Hot Encoding for Silhouette:** The Silhouette Score for the raw K-Means was calculated on one-hot encoded features, which can sometimes artificially inflate density metrics due to the high dimensionality. However, the magnitude of improvement seen with the DL approach still strongly indicates superior performance.
- **Cold-Start Problem:** New users or those with very limited interaction history may not be accurately assigned to existing clusters. A strategy for handling cold-start users (e.g., initial broad targeting followed by re-clustering) would be necessary in a production environment.
- **Static Clusters:** The clusters identified are static based on the provided dataset. User behavior and preferences evolve, necessitating regular re-training and re-evaluation of the clustering model and associated budget allocations. The optimal cadence for retraining needs to be determined.
- **Assumptions for Revenue Lift:** The monetary impact calculation relies on hypothetical values for total impressions and revenue per click. These figures should be replaced with actual business metrics for a precise financial projection.

Next Steps & Deployment Plan

To operationalize these findings and integrate them into a real-time bidding system, the following steps are recommended:

1. **Productionize the Autoencoder and K-Means Model:**
 - **Model Export:** Save the trained Autoencoder (specifically the Encoder part) and the K-Means model (centroid vectors) in a production-ready format (e.g., TensorFlow SavedModel, pickle).
 - **Inference Service:** Deploy the Encoder as a low-latency inference service (e.g., using TensorFlow Serving, FastAPI) capable of transforming new user data into latent features in real-time.
 - **Cluster Assignment:** Implement a real-time logic that takes latent features from new users and assigns them to the closest K-Means cluster.
2. **Integrate with Ad Bidding Platform:**

- **API Integration:** Connect the cluster assignment service with the ad bidding platform's API.
 - **Dynamic Bidding Rules:** Configure the bidding platform to apply different bid floors, creative sets, and targeting parameters based on the assigned user cluster.
 - **Budget Allocation Logic:** Implement the recommended budget allocation percentages directly within the ad platform's campaign settings, ensuring higher budget allocation to high-performing clusters.
3. **A/B Testing and Monitoring:**
- **Controlled Rollout:** Conduct A/B tests to validate the projected revenue lift against a control group using the previous allocation strategy.
 - **Performance Dashboards:** Establish real-time dashboards to monitor key metrics (CTR, Conversion Rate, CPC, CPA, Revenue) for each cluster and overall.
 - **Alerting:** Set up alerts for significant deviations in cluster performance or model degradation.
4. **Continuous Improvement & Maintenance:**
- **Data Drift Detection:** Monitor for changes in user behavior or data distribution that could impact model performance.
 - **Re-training Pipeline:** Develop an automated pipeline for periodic re-training of the Autoencoder and K-Means models using fresh data to adapt to evolving user patterns.
 - **Feature Engineering:** Continuously explore new features or data sources that could further enhance segmentation quality.